00 공통
공통 헤더 · 별칭 · 상수 **02**
01 수론 · 조합
Modint **02**
Dynamic Modint **02**
소수 체 **02**
확장 유클리드 **03**
중국인의 나머지 정리 **03**
이산 로그 **03**
Miller-Rabin · Pollard Rho **03**
선형 체 · phi · mu **04**
조합 전처리 **04**
Lucas **04**
02 분리 집합
Union-Find **05**
컴포넌트 값 **05**
Weighted Union-Find **05**
03 최단 경로
Dijkstra **05**
Floyd-Warshall **06**
SPFA **06**
04 그래프 · 트리
LCA **07**
LCA 경로 집계 **07**
Virtual Tree **08**
이진 점프 테이블 **08**
MST **08**
SCC **09**
SCC 축약 DAG **09**
2-SAT **09**
이분 매칭 **10**
BCC **10**
Hungarian **10**
05 유량
Dinic **10**
Min-Cost Max-Flow **11**
06 구간 자료구조
Segment Tree **12**
Lazy Segment Tree **12**
HLD **13**
HLD 경로 연산 **14**
Merge Sort Tree **14**
가환군 정책 **15**
Persistent Segment Tree **15**
PST 버전 차이 **15**
Fenwick Tree **16**
2D Fenwick Tree **16**
정·역방향 집계 어댑터 **16**
07 트라이 · 문자열
문자 집합 **16**
Trie **17**
XOR Trie **17**
Aho-Corasick **18**
KMP **19**
Suffix Array · LCP **19**
Manacher **20**
Z Algorithm **20**
이중 Rolling Hash **20**
08 기하 · 선형대수
CCW **20**
편각 정렬 **21**
선분 교차 **21**
유리수 **21**
Convex Hull **21**
모듈러 행렬 **22**
다각형 넓이 **22**
직선 방정식 · 교점 **22**
점의 다각형 포함 여부 **22**
Rotating Calipers **22**
벡터 연산 **23**
GF(2) 선형대수 **23**
XOR Basis **23**
09 다항식
FFT **23**
10 탐색 · **DP** · 오프라인
정수·실수 이분 탐색 **24**
정수·실수 삼분 탐색 **24**
LCS **24**
Mo’s Algorithm **24**
Convex Hull Trick **25**
Parallel Binary Search **25**

00 공통 헤더 · 별칭 · 상수

GNU C++23. 필요한 클래스와 policy를 함께 복사한다.

```
#include <bits/stdc++.h>
#define fastio cin.tie(0), cout.tie(0), ios::sync_with_stdio(0);
#define all(x) (x).begin(), (x).end()
#define x first
#define y second
using namespace std; using ll = long long; using u64 = uint64_t;
using ld = long double; using pld = pair<ld, ld>;
using i128 = __int128_t; using f128 = __float128;
using pll = pair<ll, ll>; using tll = tuple<ll, ll, ll>;
constexpr ll INF = 0x3f3f3f3f3f3f3f3f; constexpr ll MINF = 0xc0c0c0c0c0c0c0c0;
```

01 Modint / _mint

```
template <u64 mod> requires (mod > 0)
class _mint{ private: using mint = _mint;
  using u128 = __uint128_t;
public: u64 v;
  template <integral T = int>
  _mint(T v = 0) : v(norm((i128)v)){}
  static constexpr u64 get_mod(){ return mod; }
  static u64 norm(i128 x){ x %= mod; return (u64)(x < 0 ? x + mod : x); }
  static tuple <u64, i128, i128> gcd(u64 a, u64 b){ if(!b) return {a, 1, 0};
    auto [g, x, y] = gcd(b, a % b); return {g, y, x - (a / b) * y}; }
  mint inv() const{ assert(v); auto [g, x, y] = gcd(v, mod);
    assert(g == 1); return mint(x); }
  mint pow(u64 e) const{ mint a = *this, ret(1);
    while(e){ if(e & 1) ret *= a; a *= a; e >>= 1;
    }
    return ret; }
  mint& operator+=(const mint& a){ u64 b = a.v; if(v >= mod - b) v -= mod - b;
    else v += b; return *this; }
  mint& operator-=(const mint& a){ u64 b = a.v; if(v < b) v += mod - b;
    else v -= b; return *this; }
  mint& operator*=(const mint& a){ v = (u64)((u128)v * a.v % mod); return *this; }
  mint& operator/=(const mint& a){ return *this *= a.inv(); }
  friend mint operator+(mint a, const mint& b){ return a += b; }
  friend mint operator-(mint a, const mint& b){ return a -= b; }
  friend mint operator*(mint a, const mint& b){ return a *= b; }
  friend mint operator/(mint a, const mint& b){ return a /= b; }
  friend bool operator==(const mint& a, const mint& b){ return a.v == b.v; }
  mint operator+() const{ return *this; }
  mint operator-() const{ auto ret = *this; ret.v = v ? mod - v : 0; return ret; }
  friend ostream& operator<<(ostream& out, const mint& a){ return out << a.v; }
  friend istream& operator>>(istream& in, mint& a){ ll v; if(in >> v) a = v; return in; }
};
```

01 Dynamic Modint / _dynamic_mint

```
class _dynamic_mint{ private: using mint = _dynamic_mint;
  using u128 = __uint128_t;
  template <class T> static constexpr bool arg = integral<T> || same_as<T, mint>;
  template <class A, class B> static constexpr bool args = arg<A> && arg<B> &&
↳   (same_as<A, mint> || same_as<B, mint>);
  static u64 val(i128 a, u64 mod){ return norm(a, mod); }
  static u64 val(const mint& a, u64 mod){ assert(a.mod == mod); return a.v; }
  template <class A, class B>
  static mint cast(const A& a, const B& b){ if constexpr(same_as<A, mint>) return a;
```

Dynamic Modint / _dynamic_mint (계속)

```
    else return mint((i128)a, b.mod); }
public: u64 mod = 1, v = 0; _dynamic_mint() = default;
  _dynamic_mint(i128 v, u64 mod) : mod(mod), v(norm(v, mod)){}
  static u64 norm(i128 x, u64 m){ x %= m; return (u64)(x < 0 ? x + m : x); }
  static tuple <u64, i128, i128> gcd(u64 a, u64 b){ if(!b) return {a, 1, 0};
    auto [g, x, y] = gcd(b, a % b); return {g, y, x - (a / b) * y}; }
  mint inv() const{ assert(v); auto [g, x, y] = gcd(v, mod);
    assert(g == 1); return mint(x, mod); }
  mint pow(u64 e) const{ mint a = *this, ret(1, mod);
    while(e){ if(e & 1) ret *= a; a *= a; e >>= 1;
    }
    return ret; }
  template <class T> requires arg<T>
  mint& operator+=(const T& a){ u64 b = val(a, mod); if(v >= mod - b) v -= mod - b;
    else v += b; return *this; }
  template <class T> requires arg<T>
  mint& operator-=(const T& a){ u64 b = val(a, mod); if(v < b) v += mod - b;
    else v -= b; return *this; }
  template <class T> requires arg<T>
  mint& operator*=(const T& a){ v = (u64)((u128)v * val(a, mod) % mod); return *this; }
  template <class T> requires arg<T>
  mint& operator/=(const T& a){ return *this *= mint(val(a, mod), mod).inv(); }
  template <class A, class B> requires args<A, B>
  friend mint operator+(const A& a, const B& b){ return cast(a, b) += b; }
  template <class A, class B> requires args<A, B>
  friend mint operator-(const A& a, const B& b){ return cast(a, b) -= b; }
  template <class A, class B> requires args<A, B>
  friend mint operator*(const A& a, const B& b){ return cast(a, b) *= b; }
  template <class A, class B> requires args<A, B>
  friend mint operator/(const A& a, const B& b){ return cast(a, b) /= b; }
  template <class A, class B> requires args<A, B>
  friend bool operator==(const A& a, const B& b){ auto l = cast(a, b), r = cast(b, a);
    return l.mod == r.mod && l.v == r.v; }
  mint& operator=(i128 a){ v = norm(a, mod); return *this; }
  mint operator+() const{ return *this; }
  mint operator-() const{ auto ret = *this; ret.v = v ? mod - v : 0; return ret; }
  friend ostream& operator<<(ostream& out, const mint& a){ return out << a.v; }
  friend istream& operator>>(istream& in, mint& a){ ll v; if(in >> v) a = v; return in; }
};
```

01 소수 체 / _es

```
template <size_t n>
class _es { public: static constexpr size_t sz = (n >> 1) + 1;
  bitset <sz> bs; vector <int> prime;
  _es() { build(); }
  void build(){ prime.clear(); bs.reset(); if(n >= 2) prime.push_back(2);
    bs.set(0);
    for(int i = 3;i <= n;i += 2){ if(bs[i >> 1]) continue; prime.push_back(i);
      for(ll j = 1ll * i * i;j <= (ll)n;j += 2ll * i) bs.set(j >> 1); } }
  const vector<int>& ret() { return prime; } // 소수 벡터 반환
  template <class F>
  void it_prime(const F& f) const{ for(auto& p : prime) f(p); } // O(pi(n))
  bool is_prime(int x) const { // 소수 판별
    if(x == 2) return 1; if(x < 2 || x % 2 == 0) return 0; return !bs[x >> 1];
  } };
```

01 확장 유클리드 / _gcd

```
class _gcd { public: _gcd() {}
  tll build(ll a, ll b) { if (!b) return { a, 1, 0 }; auto [g, x, y] = build(b, a % b);
    return { g, y, x - a / b * y }; }
  ll ret(ll a, ll b) { return get<0>(build(a, b)); }
  ll inv(ll a, ll mod) { auto [g, x, y] = build(a, mod); if (!a || g != 1) return -1;
    return (x < 0 ? x + mod : x); } };
```

01 중국인의 나머지 정리 / _crt

```
class _crt{ public: enum class state{ known, inconsistent };
  class result{ public: state status;
    ll value, mod;
    bool inconsistent() const{ return status == state::inconsistent; } // O(1)
    ll get_value() const{ return value; } // O(1)
    ll get_mod() const{ return mod; } // O(1)
  }; vector <_dynamic_mint> arr;
private: result ans = {state::known, 0, 1};
  static tll gcd(ll a, ll b){ if(!b) return {a, 1, 0}; auto [g, x, y] = gcd(b, a % b);
    return {g, y, x - (a / b) * y}; }
  static result merge(const result& a, const _dynamic_mint& b){
    auto [g, cx, cy] = gcd(a.mod, (ll)b.mod); ll diff = (ll)b.v - a.value;
    if(diff % g) return {state::inconsistent, 0, 0}; i128 mod = (i128)a.mod / g * b.mod;
    ll m = b.mod / g, k = (ll)((i128)(diff / g) * cx % m); if(k < 0) k += m;
    ll v = (ll)((i128)a.value + (i128)a.mod * k); return {state::known, v, (ll)mod}; }
public: void add(ll a, ll m){ arr.emplace_back(a, m); } // O(1) amortized
  void add(pll a){ arr.emplace_back(a.x, a.y); } // O(1) amortized
  void add(const _dynamic_mint& a){ arr.push_back(a); } // O(1) amortized
  void clear(){ arr.clear(); ans = {state::known, 0, 1}; } // O(n)
  void build(){ // O(n log M)
    ans = {state::known, 0, 1};
    for(const auto& i : arr){ ans = merge(ans, i); if(ans.status != state::known) break;
    } }
  const result& ret() const{ return ans; } // O(1)
};
```

01 이산 로그 / _dlog

```
class _dlog{ public: map<ll, ll> num; vector <ll> prime;
  _dlog(){};
  void factor(ll p){ // 소인수분해
    prime.clear();
    for(ll i = 2;i <= p / i;i++){ if(p % i) continue; prime.push_back(i);
      while(p % i == 0) p /= i; }
    if(p > 1) prime.push_back(p); }
  ll phi(ll p){ factor(p); ll ret = p; for(auto& i : prime) ret /= i, ret *= i - 1;
    return ret; }
  ll sqrt(ll num){ ll lo = 0, hi = num;
    while(hi > lo){ ll mid = midpoint(hi, lo); if((i128)mid * mid < num) lo = mid;
      else hi = mid - 1; }
    return lo; }
  ll pow(ll a, ll b, ll p){ ll ret = 1 % p; a %= p; if(a < 0) a += p;
    while(b){ if(b & 1) ret = (i128)ret * a % p; a = (i128)a * a % p; b >>= 1;
    }
    return ret; }
  ll ret(ll a, ll b, ll p) { // a^x = b (mod p), ret x
    b %= p; if(b < 0) b += p; ll mx = sqrt(phi(p)) + 1; num.clear();
    ll inv = pow(a, p - 2, p), cur = b;
    for(ll i = 0;i < mx;i++) { if(num.find(cur) == num.end()) num[cur] = i;
```

이산 로그 / _dlog (계속)

```
      cur = (i128)cur * inv % p; }
    ll step = pow(a, mx, p);
    cur = 1; for(ll i = 0;i < mx;i++){
      if(num.find(cur) != num.end()) return i * mx + num[cur];
      cur = (i128)cur * step % p; }
    return -1; } };
```

01 Miller-Rabin · Pollard Rho / _pr

```
class _pr{ public: using u128 = __uint128_t; mt19937_64 rnd;
  _pr() : rnd((u64)chrono::steady_clock::now().time_since_epoch().count()){}
  u64 mul_mod(u64 a, u64 b, u64 mod){ return (u128)a * b % mod; }
  u64 pow_mod(u64 a, u64 b, u64 mod){ u64 ret = 1; a %= mod;
    while(b){ if(b & 1) ret = mul_mod(a, ret, mod); a = mul_mod(a, a, mod);
      b >>= 1ll; }
    return ret; }
  bool isprime(u64 n){ if(n < 2) return 0;
    for(u64 i : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL,
↳     31ULL, 37ULL}){
      if(n == i) return 1; if(n % i == 0) return 0; }
    u64 d = n - 1, s = 0; while((d & 1) == 0) d >>= 1ll, ++s;
    auto chk = [&](u64 a) -> bool{ if(a % n == 0) return 0; u64 x = pow_mod(a, d, n);
      if(x == 1 || x == n - 1) return 0;
      for(u64 r = 1;r < s;r++){ x = mul_mod(x, x, n); if(x == n - 1) return 0;
      }
      return 1; };
    for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL,
↳     1795265022ULL}) {
      if (chk(a)) return 0; }
    return 1; }
  u64 f(u64 x, u64 c, u64 mod){ return ((u128)mul_mod(x, x, mod) + c) % mod; }
  u64 pollard(u64 n){ // 임의의 소인수 반환
    for(u64 i : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL,
↳     31ULL, 37ULL}){
      if(n % i == 0) return i; }
    uniform_int_distribution<u64> rx(2, n - 2), rc(1, n - 1);
    while(1){ u64 a = rx(rnd), b = a; u64 c = rc(rnd), g = 1;
      while(g == 1){ a = f(a, c, n); b = f(f(b, c, n), c, n);
        u64 diff = (a > b) ? (a - b) : (b - a); g = gcd(diff, n); }
      if(g != n) return g; } }
  void dnc(u64 n, vector<u64>& ret){ if(n == 1) return;
    if(isprime(n)){ ret.push_back(n); return;
    }
    u64 p = pollard(n); dnc(p, ret); dnc(n / p, ret); }
  vector <u64> factorize(u64 n){ // 소인수분해
    vector<u64> ret; dnc(n, ret); sort(all(ret));
    return ret; }
  vector <pair<u64, int>> factorize_cnt(u64 n){ auto f = factorize(n);
    vector<pair<u64, int>> ret;
    for(u64& p : f){ if(ret.empty() || ret.back().x != p) ret.push_back({p, 1});
      else ret.back().y++; }
    return ret; }
  u64 phi(u64 n){ if(!n) return 0; auto fc = factorize_cnt(n);
    u128 ret = n; for(auto& [p, c] : fc) ret = ret / p * (p - 1); return (u64)ret;
  }
  int mobius(u64 n){ if(n == 0 || n == 1) return n; auto fc = factorize_cnt(n);
    for(auto& [p, c] : fc) if(c >= 2) return 0; return fc.size() % 2 ? -1 : 1; }
```

Miller-Rabin · Pollard Rho / _pr (계속)

```
  u64 divisor_num(u64 n){ if(!n) return 0; auto fc = factorize_cnt(n);
    u128 ret = 1; for(auto& [p, c] : fc) ret *= c + 1; return (u64)ret;
  }
  u64 divisor_sum(u64 n){ if(!n) return 0; auto fc = factorize_cnt(n);
    u128 ret = 1;
    for(auto& [p, c] : fc){ u128 now = 1, cur = 1;
      for(int i = 0;i < c;i++){ cur *= p; now += cur;
      }
      ret *= now; }
    return (u64)ret; } };
```

01 선형 체 · phi · mu / _ls

```
class _ls{ public: vector <int> lp, prime, vphi, vmu; int n = 0, flag = 0;
  _ls(){}
  _ls(int n, int flag = 0) : n(n), flag(flag){ build(); }
  void build(){ prime.clear(); vphi.clear(); vmu.clear(); lp.assign(n + 1, 0);
    if(n >= 3) prime.reserve((int)(n / log((ld)n)) + 10);
    if(flag & 1) { vphi.assign(n + 1, 0); if(n >= 1) vphi[1] = 1; }
    if(flag & 2) { vmu.assign(n + 1, 0); if(n >= 1) vmu[1] = 1; }
    for(int i = 2;i <= n;i++){ if(!lp[i]){ lp[i] = i;
        prime.push_back(i); if(flag & 1) vphi[i] = i - 1; if(flag & 2) vmu[i]  = -1;
      }
      for(auto& p : prime){ ll x = 1ll * p * i; if(x > n) break;
        lp[x] = p;
        if(i % p == 0){ if(flag & 1) vphi[x] = vphi[i] * p; if(flag & 2) vmu[x] = 0;
          break; } else{
          if(flag & 1) vphi[x] = vphi[i] * (p - 1); if(flag & 2) vmu[x] = -vmu[i]; }
      } } }
  bool is_prime(int x) const{ return x >= 2 && lp[x] == x; }
  vector <ll> factorize(int x){ // 소인수분해
    vector <ll> ret;
    while(x != 1){ ret.push_back(lp[x]); x /= lp[x];
    }
    return ret; }
  vector <pll> factorize_cnt(int x){ // p^c 꼴로 소인수분해
    vector <pll> ret;
    while(x != 1){ ll p = lp[x], c = 0; while(x % p == 0) x /= p, c++;
      ret.push_back({p, c}); }
    return ret; }
  ll divisor_num(int x){ // 약수의 개수
    ll ret = 1;
    while(x != 1){ ll p = lp[x], c = 0; while(x % p == 0) x /= p, c++;
      ret *= c + 1; }
    return ret; }
  ll divisor_sum(int x){  // 약수의 합
    if(x == 0 || x == 1) return x; ll ret = 1;
    while(x != 1){ ll p = lp[x], sum = 1, cur = 1;
      while(x % p == 0){ x /= p; cur *= p;
        sum += cur; }
      ret *= sum; }
    return ret; }
  vector <ll> divisor(int x){ // 약수 벡터 반환
    if(!x) return {}; if(x == 1) return {1}; auto fc = factorize_cnt(x);
    vector <ll> ret = {1};
    for(auto& [p, c] : fc){ ll mul = 1, sz = ret.size();
      for(ll k = 1;k <= c;k++){ mul *= p;
```

선형 체 · phi · mu / _ls (계속)

```
        for(int i = 0;i < sz;i++) ret.push_back(ret[i] * mul); } }
    sort(all(ret)); return ret; }
  ll phi(int x) const { return vphi[x]; } // 오일러 피
  ll mobius(int x) const { return vmu[x]; } // 뫼비우스
};
```

01 조합 전처리 / _comb

_mint 필요 . mod 는 소수 , 전처리 상한은 mod 미만 .

```
template <u64 mod = 998244353>
class _comb{ // mod == prime
public: using mint = _mint<mod>;
private: vector <mint> fac, ifac; int n;
public: _comb(int n = 0){ build(n); } // O(n + log mod)
  void build(int n){ // O(n + log mod)
    assert(n >= 0 && (u64)n < mod); this->n = n; fac.resize(n + 1); ifac.resize(n + 1);
    fac[0] = 1; for(int i = 1;i <= n;i++) fac[i] = fac[i - 1] * i;
    ifac[n] = fac[n].pow(mod - 2);
    for(int i = n - 1;i >= 0;i--) ifac[i] = ifac[i + 1] * (i + 1); }
  u64 comb(int n, int r) const{ // O(1)
    if(n < 0 || r < 0 || n < r) return 0; return (fac[n] * ifac[r] * ifac[n - r]).v; }
  u64 perm(int n, int r) const{ // O(1)
    if(n < 0 || r < 0 || n < r) return 0; return (fac[n] * ifac[n - r]).v; }
  u64 multicomb(int n, int r) const{ // O(1)
    if(n < 0 || r < 0) return 0; if(!n) return !r; ll total = (ll)n + r - 1;
    assert(total <= this->n); return comb((int)total, r); }
  u64 factorial(int n) const{ return fac[n].v; } // O(1)
  u64 inv_factorial(int n) const{ return ifac[n].v; } // O(1)
};
```

01 Lucas / _lucas

_dynamic_mint 필요 . mod 는 소수 , 전처리 O(mod).

```
class _lucas{ // mod == prime
private: int mod = 0; vector <int> fac, ifac;
  int cal(int n, int r) const{ // O(1)
    if(n < r) return 0; return (ll)fac[n] * ifac[r] % mod * ifac[n - r] % mod; }
public: _lucas() = default;
  _lucas(int mod){ build(mod); } // O(mod)
  void build(int mod){ // O(mod)
    assert(mod >= 2); this->mod = mod; fac.resize(mod); ifac.resize(mod); fac[0] = 1;
    for(int i = 1;i < mod;i++) fac[i] = (ll)fac[i - 1] * i % mod;
    ifac[mod - 1] = _dynamic_mint(fac[mod - 1], mod).pow(mod - 2).v;
    for(int i = mod - 2;i >= 0;i--) ifac[i] = (ll)ifac[i + 1] * (i + 1) % mod; }
  int get_mod() const{ return mod; } // O(1)
  int comb(ll n, ll r) const{ // O(log_mod(n))
    if(n < r || n < 0 || r < 0) return 0; if(!n || !r || n == r) return 1;
    return (ll)cal(n % mod, r % mod) * comb(n / mod, r / mod) % mod; }
  int perm(ll n, ll r) const{ // O(1)
    if(n < 0 || r < 0 || n < r || r >= mod) return 0; int rem = n % mod;
    if(rem < r) return 0; return (ll)fac[rem] * ifac[rem - r] % mod; }
  int multicomb(ll n, ll r) const{ // O(log_mod(n + r))
    if(n < 0 || r < 0) return 0; if(!n) return !r; i128 total = (i128)n + r - 1;
    assert(total <= LLONG_MAX); return comb((ll)total, r); }
};
```

02 Union-Find / _uf

```
class _uf{ private: bool base;
  vector<int> p; int n, cc;
protected: bool merge_internal(int& a, int& b){ // O(alpha(n)) amortized
    a = find(a); b = find(b); if(a == b) return false; if(p[a] > p[b]) swap(a, b);
    p[a] += p[b]; p[b] = a; cc--; return true; }
public: _uf(int n = 0, bool base = 1){ clear(n, base); } // O(n)
  void clear(int n, bool base = 1){ // O(n)
    this->n = n; this->base = base; cc = n - base + 1; p.assign(n + 1, -1); }
  int find(int x){ // O(alpha(n)) amortized
    if(p[x] < 0) return x; return p[x] = find(p[x]); }
  bool merge(int a, int b){ return merge_internal(a, b); } // O(alpha(n)) amortized
  int count() const{ return cc; } // O(1)
  int size(int x){ return -p[find(x)]; } // O(alpha(n)) amortized
  bool root(int x) const{ return p[x] < 0; } // O(1)
  bool same(int a, int b){ return find(a) == find(b); } // O(alpha(n)) amortized
  template<class F>
  void it_root(const F& f) const{ // O(n)
    for(int i = base;i <= n;i++) if(root(i)) f(i); }
  vector<vector<int>> groups(){ // O(n * alpha(n))
    vector<int> id(n + 1, -1); vector<vector<int>> ret; ret.reserve(cc);
    it_root([&](int r){ id[r] = (int)ret.size(); ret.emplace_back();
      ret.back().reserve(-p[r]); });
    for(int i = base;i <= n;i++) ret[id[find(i)]].push_back(i); return ret; }
};
```

02 컴포넌트 값 / _uf_value

_uf 필요 . uf_policy 포함 .

```
class uf_policy{ public:
  class node{};
  static node leaf(){ return {}; }
  static void merge(node& a, node& b){} };
template <class policy = uf_policy>
class _uf_value : public _uf{ public: using node = typename policy::node;
private: bool base; vector<node> arr;
  node leaf() const{ return policy::leaf(); }
public:
  _uf_value(int n = 0, bool base = 1) : _uf(n, base), base(base), arr(n + 1, leaf()){}
↳   // O(n)
  void clear(int n, bool base = 1){ // O(n)
    _uf::clear(n, base); this->base = base; arr.assign(n + 1, leaf()); }
  void make(span<const node> v){ // O(n)
    vector<node> tmp(v.size(), leaf());
    for(int i = base;i < (int)v.size();i++) tmp[i] = v[i];
    _uf::clear(v.empty() ? base - 1 : (int)v.size() - 1, base); arr.swap(tmp); }
  bool merge(int a, int b){ // O(alpha(n)) amortized + policy::merge
    if(!merge_internal(a, b)) return false; policy::merge(arr[a], arr[b]); return true;
  }
  node& get(int x){ return arr[find(x)]; } // O(alpha(n)) amortized
  void set(int x, const node& v){ arr[find(x)] = v; } // O(alpha(n)) amortized
  template<class F>
  void it_component(const F& f){ // O(n)
    it_root([&](int root){ f(root, arr[root]); }); } };
// WEIGHTED UNION FIND
```

02 Weighted Union-Find / _wuf

```
class _wuf{ private: bool base;
  vector<int> p; vector<ll> d; vector<char> dis; int n, cc;
public: enum class state{ disconnected, inconsistent, known };
  class result{ public: state status;
    ll value;
    bool disconnected() const{ return status == state::disconnected; } // O(1)
    bool inconsistent() const{ return status == state::inconsistent; } // O(1)
    ll get_value() const{ return value; } // O(1)
  };
  _wuf(int n = 0, bool base = 1){ clear(n, base); } // O(n)
  void clear(int n, bool base = 1){ // O(n)
    this->n = n; this->base = base; cc = n - base + 1;
    p.assign(n + 1, -1); d.assign(n + 1, 0); dis.assign(n + 1, 0); }
  pair<int, ll> find(int num){ // O(alpha(n)) amortized
    if(p[num] < 0) return {num, 0ll}; auto [pa, diff] = find(p[num]);
    p[num] = pa; d[num] += diff; return {p[num], d[num]}; }
  void merge(int a, int b, ll c){ // O(alpha(n)) amortized
    auto [pa, da] = find(a); auto [pb, db] = find(b);
    if(pa == pb){ dis[pb] |= ((i128)db - da != c); return; }
    i128 delta = (i128)db - da - c;
    if(p[pa] < p[pb]){ swap(pa, pb); delta = -delta; }
    p[pb] += p[pa]; p[pa] = pb; d[pa] = (ll)delta; dis[pb] |= dis[pa]; cc--; }
  result diff(int a, int b){ // O(alpha(n)) amortized
    auto [pa, da] = find(a); auto [pb, db] = find(b);
    if(pa != pb) return {state::disconnected, 0};
    if(dis[pa]) return {state::inconsistent, 0}; return {state::known, db - da}; }
  bool same(int a, int b){ return find(a).x == find(b).x; } // O(alpha(n)) amortized
  int count() const{ return cc; } // O(1)
  int size(int x){ return -p[find(x).x]; } // O(alpha(n)) amortized
  bool root(int x) const{ return p[x] < 0; } // O(1)
  template<class F>
  void it_root(const F& f) const{ // O(n)
    for(int i = base;i <= n;i++) if(root(i)) f(i); }
  vector<vector<int>> groups(){ // O(n * alpha(n))
    vector<int> id(n + 1, -1); vector<vector<int>> ret; ret.reserve(cc);
    it_root([&](int r){ id[r] = (int)ret.size(); ret.emplace_back();
      ret.back().reserve(-p[r]); });
    for(int i = base;i <= n;i++) ret[id[find(i).x]].push_back(i); return ret; }
};
```

03 Dijkstra · sp_policy / _dij

간선 가중치가 음수가 없을 때 사용 .

```
struct sp_policy{ using dist = ll; using cost = ll;
  static dist add(const dist& d, const cost& w){ // dist + cost
    return d + w; }
  static bool leq(const dist& a, const dist& b){ // dist a <= dist b
    return a <= b; }
  static dist inf(){ // ll -> INF, pll -> {INF, INF}, tll -> {INF, INF, INF}
    return INF; }
  static dist zero(){ // ll -> 0, pll -> {0, 0}, tll -> {0, 0, 0}
    return 0; } };
template<class policy = sp_policy>
class _dij { bool base;
public: using dist = typename policy::dist; using cost = typename policy::cost;
private:
  struct edge{ cost w; int nxt; }; struct node{ dist d; int v; };
```

Dijkstra · sp_policy / _dij (계속)

```
  struct cmp{ bool operator()(const node& a, const node& b) const{ return
↳   _dij::less(b.d, a.d); } };
  vector<vector<edge>> adj; int n; bool built = 0; vector <dist> d; vector<int> pre;
  priority_queue<node, vector<node>, cmp> pq;
  void cal(){ built = 1;
    while (!pq.empty()) { auto [cd, cur] = pq.top(); pq.pop();
      if(!eq(cd, d[cur])) continue;
      for(const auto& i : adj[cur]) { auto& [w, nxt] = i; dist nd = policy::add(cd, w);
        if(policy::leq(d[nxt], nd)) continue; d[nxt] = nd; pre[nxt] = cur;
        pq.push({ nd, nxt }); } }
  }
  void reset(int n){ pre.assign(n + 1, -1); d.assign(n + 1, policy::inf()); }
  void chk(int x) const{ assert(x >= base && x <= n); }
public:
  static bool less(const dist& a, const dist& b){ return policy::leq(a, b) &&
↳   !policy::leq(b, a); } // O(1)
  static bool eq(const dist& a, const dist& b){ return policy::leq(a, b) &&
↳   policy::leq(b, a); }   // O(1)
  _dij(int n = 0, bool base = 1){ clear(n, base); } // O(n)
  void clear(int n, bool base = 1){ // O(n + m)
    this->n = n; this->base = base; adj.assign(n + 1, {}); reset(n);
    built = 0; while(!pq.empty()) pq.pop(); }
  void addsol(int st, int en, cost c){ // O(1)
    assert(base <= st && st <= n && base <= en && en <= n); built = 0;
    adj[st].push_back({ c,en }); }
  void add(int st, int en, cost c){ // O(1)
    addsol(st, en, c); addsol(en, st, c); }
  void build(int source){ // O((n + m) log n)
    assert(base <= source && source <= n); while(!pq.empty()) pq.pop();
    reset(n); d[source] = policy::zero(); pq.push({ d[source], source }); cal(); }
  void build(const vector <int>& source){ // O((n + m) log n)
    reset(n);
    for(const auto& i : source){ assert(base <= i && i <= n); d[i] = policy::zero();
      pq.push({ d[i], i }); } cal(); }
  dist ret(int x) const{ // O(1)
    chk(x); return d[x]; }
  bool reachable(int x) const{ // O(1)
    chk(x); return !eq(d[x], policy::inf()); }
  template<class F>
  void it_path(int x, const F& f) const{ // O(n)
    chk(x); assert(reachable(x)); vector<int> ret;
    for(int cur = x; cur != -1; cur = pre[cur]) ret.push_back(cur); reverse(all(ret));
    for(const auto& v : ret) f(v); } };
```

03 Floyd-Warshall / _floyd

sp_policy 필요.

```
template <class policy = sp_policy>
class _floyd { bool base;
public: using dist = typename policy::dist; using cost = typename policy::cost;
private: vector <vector<dist>> d, input; vector <vector<int>> nxt; int n; bool built = 0;
  static bool less(const dist& a, const dist& b){ return policy::leq(a, b) &&
↳   !policy::leq(b, a); }
  static bool eq(const dist& a, const dist& b){ return policy::leq(a, b) &&
↳   policy::leq(b, a); }
  void chk(int st, int en) const{ assert(st >= base && st <= n); assert(en >= base && en
↳   <= n); }
```

Floyd-Warshall / _floyd (계속)

```
public: _floyd(int n = 0, bool base = 1){ clear(n, base); } // O(n^2)
  void clear(int n, bool base = 1){ // O(n^2)
    this->n = n; this->base = base; built = 0;
    input.assign(n + 1, vector<dist>(n + 1, policy::inf())); d = input;
    nxt.assign(n + 1, vector<int>(n + 1, 0));
    for(int i = base; i <= n; i++){ input[i][i] = d[i][i] = policy::zero();
      nxt[i][i] = i; } }
  void add(int st, int en, cost c) { // O(1)
    addsol(st, en, c); addsol(en, st, c); }
  void addsol(int st, int en, cost c) { // O(1)
    assert(base <= st && st <= n && base <= en && en <= n); built = 0;
    if(!less(c, input[st][en])) return; input[st][en] = c; }
  void build() { // O(n^3)
    d = input; nxt.assign(n + 1, vector<int>(n + 1, 0));
    for(int i = base; i <= n; ++i) for(int j = base; j <= n; ++j)
      if(!eq(d[i][j], policy::inf())) nxt[i][j] = j;
    built = 1;
    for (int k = base; k <= n; k++) { for (int i = base; i <= n; i++) {
        for (int j = base; j <= n; j++) {
          if(eq(d[i][k], policy::inf()) || eq(d[k][j], policy::inf())) continue;
          dist nd = policy::add(d[i][k], d[k][j]);
          if (policy::leq(d[i][j], nd)) continue; d[i][j] = nd; nxt[i][j] = nxt[i][k]; }
      } } }
  dist ret(int st, int en) const{ // O(1)
    chk(st, en); return d[st][en]; }
  bool reachable(int st, int en) const{ // O(1)
    chk(st, en); return !eq(d[st][en], policy::inf()); }
  template<class F>
  void it_path(int st, int en, const F& f) const{ // O(n)
    chk(st, en); assert(reachable(st, en));
    for(int cur = st;; cur = nxt[cur][en]){ f(cur); if(cur == en) break;
    } } };
```

03 SPFA / _spfa

sp_policy 필요. 최악 시간 복잡도 O(VE).

```
template <class policy = sp_policy>
class _spfa { bool base;
public: using dist = typename policy::dist; using cost = typename policy::cost;
private:
  struct edge{ cost w; int nxt; }; vector <int> pre, cnt; deque <int> q;
  vector <dist> d; vector <bool> in, cy;
  vector <vector <edge>> adj; int n; bool built = 0;
  static bool less(const dist& a, const dist& b){ return policy::leq(a, b) &&
↳   !policy::leq(b, a); }
  static bool eq(const dist& a, const dist& b){ return policy::leq(a, b) &&
↳   policy::leq(b, a); }
  void cal_cy(int x){ if(cy[x]) return; cy[x] = 1; deque <int> q; q.push_back(x);
    while(!q.empty()){ int cur = q.front(); q.pop_front();
      for(auto& e : adj[cur]){ if(cy[e.nxt]) continue; cy[e.nxt] = 1; q.push_back(e.nxt);
      } } }
  bool cal(){ built = 1; bool ret = 1;
    while (!q.empty()) { int cur = q.front(); q.pop_front();
      in[cur] = 0; dist cd = d[cur]; if(cy[cur]) continue;
      for (auto& e : adj[cur]) { auto [co, nxt] = e; if(cy[nxt]) continue;
        dist nd = policy::add(cd, co); if (policy::leq(d[nxt], nd)) continue;
        d[nxt] = nd; pre[nxt] = cur; cnt[nxt] = cnt[cur] + 1;
```

SPFA / _spfa (계속)

```
        if(cnt[nxt] >= n - base + 1){ ret = 0; cal_cy(nxt); continue;
        }
        if(!in[nxt]){ in[nxt] = 1; q.push_back(nxt); } } }
    return ret; }
  void reset(int n){ pre.assign(n + 1, -1); cnt.assign(n + 1, 0);
    d.assign(n + 1, policy::inf()); q.clear();
    in.assign(n + 1, false); cy.assign(n + 1, false); }
  void chk(int x) const{ assert(x >= base && x <= n); }
public: _spfa(int n = 0, bool base = 1){ clear(n, base); } // O(n)
  void clear(int n, bool base = 1){ // O(n + m)
    this->n = n; this->base = base; built = 0; reset(n); adj.assign(n + 1, {}); }
  void addsol(int st, int en, cost c) { // O(1)
    assert(base <= st && st <= n && base <= en && en <= n); built = 0;
    adj[st].push_back({ c, en }); }
  void add(int st, int en, cost c) { // O(1)
    addsol(st, en, c); addsol(en, st, c); }
  // cycle existed -> return 0
  bool build(int source, dist fi = policy::zero()) { // O(nm)
    assert(base <= source && source <= n); reset(n); d[source] = fi; in[source] = 1;
    q.push_back(source); return cal(); }
  // cycle existed -> return 0
  bool build(const vector <int>& source, dist fi = policy::zero()) { // O(nm)
    reset(n);
    for (const auto& st : source) { assert(base <= st && st <= n);
      d[st] = fi; in[st] = 1; q.push_back(st); }
    return cal(); }
  dist ret(int x) const{ // O(1)
    chk(x); return d[x]; }
  bool reachable(int x) const{ // O(1)
    chk(x); return !is_cycle(x) && !eq(d[x], policy::inf()); }
  bool is_cycle(int x) const{ // O(1)
    chk(x); return cy[x]; }
  template<class F>
  void it_path(int x, const F& f) const{ // O(n)
    chk(x); assert(reachable(x)); assert(!is_cycle(x)); vector <int> ret;
    for(int cur = x; cur != -1; cur = pre[cur]) ret.push_back(cur);
    for(auto it = ret.rbegin(); it != ret.rend(); ++it) f(*it); } };
```

04 LCA / _lca

```
class _lca { protected: bool base;
  int n, lg; vector<vector<int>> p, adj; vector<int> d, comp, ord;
private: void build_comp(int root, int cid){
    stack<int> st; st.push(root); d[root] = 0; comp[root] = cid;
    while(!st.empty()){ int cur = st.top(); st.pop(); ord.push_back(cur);
      for(int nxt : adj[cur]){ if(d[nxt] != -1) continue;
        p[0][nxt] = cur; d[nxt] = d[cur] + 1; comp[nxt] = cid; st.push(nxt); }
    } }
  int shift(int cur, int diff) const{
    for(int i = lg; i >= 0; i--) if(diff & (1ll << i)) cur = p[i][cur]; return cur; }
public: _lca(int n = 0, bool base = 1){ clear(n, base); } // O(n log n)
  void clear(int n, bool base = 1){ // O(n log n)
    this->n = n; this->base = base; lg = 1; while((1ll << lg) <= n) lg++;
    p.assign(lg + 1, vector<int>(n + 1, -1));
    adj.assign(n + 1, {}); d.assign(n + 1, -1); comp.assign(n + 1, 0);
    ord.clear(); ord.reserve(n + 1); }
  void add(int a, int b){ addsol(a, b); addsol(b, a); } // O(1)
```

LCA / _lca (계속)

```
  void addsol(int a, int b){ adj[a].push_back(b); } // O(1)
  void build(){ // O(n log n)
    for(auto& row : p) fill(all(row), -1);
    fill(all(d), -1); fill(all(comp), 0); ord.clear(); int cid = 0;
    for(int i = base; i <= n; i++) if(d[i] == -1) build_comp(i, ++cid);
    for(int i = 1; i <= lg; i++) for(int v = base; v <= n; v++){ int mid = p[i - 1][v];
      if(mid != -1) p[i][v] = p[i - 1][mid]; } }
  int component(int v) const{ return comp[v]; } // O(1)
  bool same_comp(int a, int b) const{ return comp[a] == comp[b]; } // O(1)
  int depth(int v) const{ return d[v]; } // O(1)
  int parent(int v) const{ return p[0][v]; } // O(1)
  int ancestor(int v, int k) const{ // O(log n)
    if(k < 0 || k > d[v]) return -1; return shift(v, k); }
  template<bool REV = true, class F>
  void it(const F& f) const{ // O(n)
    if constexpr (!REV) for(int v : ord) f(v);
    else for(auto iter = ord.rbegin(); iter != ord.rend(); ++iter) f(*iter); }
  int ret(int a, int b) const{ // O(log n)
    if(!same_comp(a, b)) return -1; if(d[a] < d[b]) swap(a, b);
    a = shift(a, d[a] - d[b]); if(a == b) return a;
    for(int i = lg; i >= 0; i--){ if(p[i][a] == p[i][b]) continue;
      a = p[i][a]; b = p[i][b]; }
    return p[0][a]; }
  int dist(int a, int b) const{ // O(log n)
    int l = ret(a, b); return l == -1 ? -1 : d[a] + d[b] - 2 * d[l]; }
  int jump(int a, int b, int k) const{ // O(log n)
    int l = ret(a, b); if(l == -1) return -1; int da = d[a] - d[l], db = d[b] - d[l];
    if(k < 0 || k > da + db) return -1;
    return k <= da ? shift(a, k) : shift(b, da + db - k); } };
```

04 LCA 경로 집계 / _lca_path

_lca 필요 . lca_policy 포함 .

```
class lca_policy { public:
  class node { public: ll v;
    node(ll v) : v(v){} };
  class cost { public: ll v;
    cost(ll v = 0) : v(v){} };
  static node id(){ return {MINF}; }
  static node lift(const cost& c){ return {c.v}; }
  static node merge(const node& a, const node& b){ return {max(a.v, b.v)}; } };
template<class policy = lca_policy>
class _lca_path : public _lca { public: using node = typename policy::node;
  using cost = typename policy::cost;
private:
  class path { public: node up, down;
  }; vector<vector<pair<cost,cost>>> weight; vector<vector<path>> arr;
  static path merge(const path& a, const path& b){
    return {policy::merge(a.up, b.up), policy::merge(b.down, a.down)}; }
public:
  _lca_path(int n = 0, bool base = 1) : _lca(n, base), weight(n + 1){} // O(n log n)
  void clear(int n, bool base = 1){ // O(n log n)
    _lca::clear(n, base); weight.assign(n + 1, {}); arr.clear(); }
  void add(int a, int b, cost co = cost()){ add(a, b, co, co); } // O(1)
  void add(int a, int b, cost ab, cost ba){ addsol(a, b, ab, ba); addsol(b, a, ba, ab);
↳   } // O(1)
  void addsol(int a, int b, cost co = cost()){ addsol(a, b, co, co); } // O(1)
```

LCA 경로 집계 / _lca_path (계속)

```
  void addsol(int a, int b, cost ab, cost ba){ // O(1)
    _lca::addsol(a, b); weight[a].push_back({ab, ba}); }
  void build(){ // O(n log n)
    _lca::build(); arr.assign(lg + 1, vector<path>(n + 1, {policy::id(), policy::id()}));
    vector<bool> used(n + 1, 0);
    for(int a = base; a <= n; a++) for(int i = 0; i < (int)adj[a].size(); i++){
      int b = adj[a][i]; if(p[0][b] != a || used[b]) continue; used[b] = 1;
      const auto& [ab, ba] = weight[a][i];
      arr[0][b] = {policy::lift(ba), policy::lift(ab)}; }
    for(int i = 1; i <= lg; i++) for(int v = base; v <= n; v++){ int mid = p[i - 1][v];
      if(mid != -1) arr[i][v] = merge(arr[i - 1][v], arr[i - 1][mid]); } }
  node query(int a, int b) const{ // O(log n)
    node left = policy::id(), right = policy::id(); int l = ret(a, b);
    if(l == -1) return left; int da = d[a] - d[l], db = d[b] - d[l];
    for(int i = lg; i >= 0; i--){ if(da & (1ll << i)){
        left = policy::merge(left, arr[i][a].up); a = p[i][a]; }
      if(db & (1ll << i)){ right = policy::merge(arr[i][b].down, right); b = p[i][b]; }
    }
    return policy::merge(left, right); } };
```

04 Virtual Tree / _vtree

원본 트리를 전처리한 뒤 필요한 정점 집합을 압축한다 .

```
class _vtree { bool base;
private: vector <vector<int>> lift; vector <ll> dist;
  vector <int> in, out, dep, used; int n, lg, seq; bool built = 0;
  bool is_ancestor(int a, int b) const{ return in[a] <= in[b] && out[b] <= out[a]; }
  void sort_node(vector <int>& node) const{ sort(all(node), [&](int a, int b){
      return in[a] < in[b]; }); }
  void clear_vt(){ for(auto& v : used) vt[v].clear(); used.clear();
  }
  int lca(int a, int b) const{ if(is_ancestor(a, b)) return a;
    if(is_ancestor(b, a)) return b;
    for(int i = lg;i >= 0;i--){ if(!is_ancestor(lift[i][a], b)) a = lift[i][a]; }
    return lift[0][a]; }
  void dfs(int cur, int pre){ in[cur] = ++seq; lift[0][cur] = pre;
    for(int i = 1;i <= lg;i++){ lift[i][cur] = lift[i - 1][lift[i - 1][cur]]; }
    for(auto& [nxt, co] : adj[cur]){ if(nxt == pre) continue; dep[nxt] = dep[cur] + 1;
      dist[nxt] = dist[cur] + co; dfs(nxt, cur); }
    out[cur] = seq; }
public: vector <vector<pair<int, ll>>> adj, vt;
  _vtree(int n = 0, bool base = 1){ clear(n, base); } // O(1)
  void clear(int n, bool base = 1){ // O(n)
    this->n = n; this->base = base; seq = 0; built = 0; lg = 1;
    while((1ll << lg) <= n) lg++; adj.assign(n + 1, {}); vt.assign(n + 1, {});
    lift.assign(lg + 1, vector<int>(n + 1, 0));
    in.assign(n + 1, 0); out.assign(n + 1, 0);
    dep.assign(n + 1, 0); dist.assign(n + 1, 0); used.clear(); }
  void add(int a, int b, ll c = 1){ adj[a].push_back({b, c}); adj[b].push_back({a, c});
    built = 0; } // O(1)
  void build(int root = -1){ // O(n log n)
    seq = 0; built = 1; clear_vt(); for(auto& row : lift) fill(all(row), 0);
    fill(all(in), 0); fill(all(out), 0); fill(all(dep), 0); fill(all(dist), 0);
    if(n < base) return; if(root == -1) root = base; assert(base <= root && root <= n);
    dfs(root, root); }
  int depth(int v) const { return dep[v]; } // O(1)
  int parent(int v) const { return lift[0][v]; } // O(1)
```

Virtual Tree / _vtree (계속)

```
  ll dist_root(int v) const { return dist[v]; } // O(1)
  int ret(int a, int b) const { return lca(a, b); } // O(log n)
  ll dist_tree(int a, int b) const{ // O(log n)
    int c = lca(a, b); return dist[a] + dist[b] - 2 * dist[c]; }
  vector <int> make(vector <int> node){ // O(k log k)
    clear_vt(); if(node.empty()) return {}; sort_node(node);
    int sz = node.size();
    for(int i = 0;i < sz - 1;i++) node.push_back(lca(node[i], node[i + 1]));
    sort_node(node); node.erase(unique(all(node)), node.end()); used = node;
    vector<int> st = {node[0]};
    for(int i = 1;i < node.size();i++){ int v = node[i];
      while(!st.empty() && !is_ancestor(st.back(), v)){ st.pop_back(); }
      vt[st.back()].push_back({v, dist[v] - dist[st.back()]}); st.push_back(v); }
    return node; } };
```

04 이진 점프 테이블 / _st

```
class _st { bool base;
private: vector <vector<i128>> sum; vector <vector<int>> lift; ll m; int n, lg;
public:
  _st(int n = 0, ll max_step = (1ll << 60), bool base = 1){ clear(n, max_step, base); };
↳   // O(n log m)
  void clear(int n, ll m = (1ll << 60), bool base = 1){ // O(n log m)
    lg = 1; this->n = n; this->base = base; this->m = m; while((1ll << lg) <= m) lg++;
    lift.assign(lg + 1, vector<int>(n + 2, n + 1));
    sum.assign(lg + 1, vector<i128>(n + 2, 1)); }
  void add(int st, int en, ll co = 1) { assert(base <= st && st <= n && base <= en && en
↳   <= n + 1); lift[0][st] = en; sum[0][st] = co; } // O(1)
  void build(){ // O(n log m)
    for(int i = 1;i <= lg;i++){ for(int j = base;j <= n;j++){ int mid = lift[i - 1][j];
        lift[i][j] = lift[i - 1][mid]; sum[i][j] = sum[i - 1][j] + sum[i - 1][mid]; }
    } }
  int next(int v) const{ return lift[0][v]; } // O(1)
  pair<int, ll> ret(int cur, ll k) const{ // O(log m)
    ll co = 0;
    for(int i = lg; i >= 0; i--){ if(k < sum[i][cur]) continue;
      co += sum[i][cur]; k -= sum[i][cur]; cur = lift[i][cur]; }
    return {cur, co}; // 도착 정점, 사용한 비용
  } };
```

04 MST / _mst

_uf 필요 . mst_policy 포함 .

```
class mst_policy{ public:
  class node{ public: ll v;
    node(ll v) : v(v){}
    node operator+(const node& other) const{ return {v + other.v}; } };
  static node leaf(){ return {0}; }
  static bool less(const node& a, const node& b){ return a.v < b.v; } };
template <class policy = mst_policy>
class _mst{ bool base;
public: using node = typename policy::node;
private: node leaf() const{ return policy::leaf(); }
  bool less(const node& a, const node& b) const{ return policy::less(a, b); }
  node result = leaf(); int n;
  class edge{ public: int s, e; node c;
  }; _uf uf; vector<edge> adj; vector<int> selected;
public: _mst(int n = 0, bool base = 1){ clear(n, base); } // O(n)
```

MST / _mst (계속)

```
  void clear(int n, bool base = 1){ // O(n)
    this->n = n; this->base = base; result = leaf();
    uf.clear(n, base); adj.clear(); selected.clear(); }
  int add(int st, int en, node c){ // O(1)
    int id = adj.size(); adj.push_back({st, en, c}); return id; }
  bool build(int components = 1){ // O(n + m log m)
    uf.clear(n, base); result = leaf(); selected.clear();
    vector<int> order(adj.size()); iota(all(order), 0);
    sort(all(order), [&](int a, int b){ if(less(adj[a].c, adj[b].c)) return true;
      if(less(adj[b].c, adj[a].c)) return false; return a < b; });
    int need = n - base + 1 - components;
    for(int id : order){ if((int)selected.size() >= need) break;
      const auto& [st, en, c] = adj[id]; if(!uf.merge(st, en)) continue;
      result = result + c; selected.push_back(id); }
    return (int)selected.size() >= need; }
  node ret() const{ return result; } // O(1)
  template<class F>
  void it_edge(const F& f) const{ // O(k)
    for(int id : selected){ const auto& [st, en, c] = adj[id]; f(id, st, en, c);
    } } };
```

04 SCC / _scc

```
class _scc{ protected: vector <vector<int>> adj, rev, group;
  vector <int> comp, ord; vector <bool> v; int n, cc; bool base;
private: void dfs1(int cur){ v[cur] = 1;
    for(auto& nxt : adj[cur]) if(!v[nxt]) dfs1(nxt); ord.push_back(cur); }
  void dfs2(int cur, int cid){ comp[cur] = cid; group[cid].push_back(cur);
    for(auto& nxt : rev[cur]) if(comp[nxt] == -1) dfs2(nxt, cid); }
public: _scc(int max_index = 0, bool base = 1){ clear(max_index, base); } // O(n)
  void clear(int max_index, bool base = 1){ // O(n)
    this->n = max_index; this->base = base;
    cc = 0; adj.assign(n + 1, {}); rev.assign(n + 1, {});
    comp.assign(n + 1, -1); group.clear(); ord.clear();
    v.assign(n + 1, false); ord.reserve(n + 1); }
  void addsol(int s, int e){ adj[s].push_back(e); rev[e].push_back(s); } // O(1)
  void build(){ // O(n + m)
    cc = 0; ord.clear(); group.clear(); fill(all(v), false); fill(all(comp), -1);
    for(int i = base;i <= n;i++) if(!v[i]) dfs1(i);
    for(int i = (int)ord.size() - 1;i >= 0;i--){
      int cur = ord[i]; if(comp[cur] != -1) continue; group.push_back({});
      dfs2(cur, cc); cc++; } }
  int group_size(int cid) const{ return (int)group[cid].size(); } // O(1)
  int component(int x) const{ return comp[x]; } // O(1)
  bool same(int a, int b) const{ return comp[a] == comp[b]; } // O(1)
  int comp_cnt() const{ return cc; } // O(1)
  int root(int cid) const{ return group[cid][0]; } // O(1)
  template <bool REV = false, class F>
  void it_cid(const F& f) const{ // O(n)
    if constexpr(!REV) for(int i = 0;i < cc;i++) f(i);
    else for(int i = cc - 1;i >= 0;i--) f(i); }
  template <bool REV = false, class F>
  void it_all(const F& f) const{ // O(n)
    it_cid<REV>([&](int cid){ for(auto& i : group[cid]) f(i); }); }
  template<class F>
  void it_group(int cid, const F& f) const{ // O(n)
    for(auto& cur : group[cid]) f(cur); } };
```

04 SCC 축약 DAG / _scc_dag

_scc 필요.

```
class _scc_dag : public _scc { private: array<vector<vector<int>>, 2> edges;
  array<vector<int>, 2> degree, order;
public: _scc_dag(int max_index = 0, bool base = 1) : _scc(max_index, base){} // O(n)
  void clear(int max_index, bool base = 1){ // O(n + m)
    _scc::clear(max_index, base); for(auto& g : edges) g.clear();
    for(auto& d : degree) d.clear(); for(auto& v : order) v.clear(); }
  void build(){ // O(n + m)
    _scc::build(); for(auto& g : edges) g.assign(cc, {});
    for(auto& d : degree) d.assign(cc, 0); vector<int> mark(cc, -1);
    for(int i = 0; i < cc; i++) for(int cur : group[i]) for(int nxt : adj[cur]){
      int j = comp[nxt]; if(i == j || mark[j] == i) continue; mark[j] = i;
      edges[0][i].push_back(j); edges[1][j].push_back(i); degree[0][j]++; degree[1][i]++;
    }
    for(int dir = 0; dir < 2; dir++){ auto pending = degree[dir];
      auto& top = order[dir]; top.clear(); top.reserve(cc);
      for(int i = 0; i < cc; i++) if(!pending[i]) top.push_back(i);
      for(size_t i = 0; i < top.size(); i++)
        for(int nxt : edges[dir][top[i]]) if(!--pending[nxt]) top.push_back(nxt); } }
  const vector<vector<int>>& dag() const{ return edges[0]; } // O(1)
  const vector<int>& ind() const{ return degree[0]; } // O(1)
  const vector<int>& outd() const{ return degree[1]; } // O(1)
  template<class F>
  void it_dag(const F& f) const{ // O(k + e)
    for(int i = 0; i < cc; i++) for(int j : edges[0][i]) f(i, j); }
  template<class F, class G>
  void it_top(const F& merge, const G& done, bool rev = true) const{ // O(k + e)
    for(int cur : order[rev]){ done(cur); for(int nxt : edges[rev][cur]) merge(cur, nxt);
    } } };
```

04 2-SAT / _2sat

_scc 필요. 리터럴은 pos(x), neg(x) 로 만든다.

```
class _2sat{ private: int n = 0, vars = 0; bool base = 1;
public: _scc scc;
  _2sat(int max_index = 0, bool base = 1){ clear(max_index, base); } // O(n)
  void clear(int max_index, bool base = 1){ // O(n)
    this->n = max_index; this->base = base; vars = n - base + 1;
    scc.clear(n + vars, base); }
  int pos(int x) const{ return x; } // O(1)
  int neg(int x) const{ return x + vars; } // O(1)
  int op(int x) const{ return x <= n ? x + vars : x - vars; } // O(1)
  void add(int a, int b){ scc.addsol(a, b); } // O(1)
  void add_imp(int a, int b){ add_or(op(a), b); } // O(1)
  void add_or(int a, int b){ add(op(a), b); add(op(b), a); } // O(1)
  void add_true(int x){ add(op(x), x); } // O(1)
  void add_false(int x){ add_true(op(x)); } // O(1)
  void add_eq(int a, int b){ add_imp(a, b); add_imp(b, a); } // O(1)
  void add_xor(int a, int b){ add_or(a, b); add_or(op(a), op(b)); } // O(1)
  void add_most1(int a, int b){ add_or(op(a), op(b)); } // O(1)
  void add_most1(int a, int b, int c){ add_most1(a, b); add_most1(b, c); add_most1(a,
↳   c); } // O(1)
  void add_least2(int a, int b, int c){ add_or(a, b); add_or(a, c); add_or(b, c); } //
↳   O(1)
  bool build(){ // O(n + m)
    scc.build();
    for(int i = base;i <= n;i++){ if(scc.same(pos(i), neg(i))) return false; }
```

2-SAT / _2sat (계속)

```
    return true; }
  template<class F>
  void it(const F& f) const{ // O(n)
    for(int i = base; i <= n + vars; i++){
      int x = (scc.component(i) > scc.component(op(i))) ? 1 : 0; f(i, x); }
  } };
```

04 이분 매칭 / _bm

```
class _bm { bool base;
private: vector<vector<int>> adj; int as, bs, result; vector<int> av, bv, dist;
  static constexpr int INF = 0x3f3f3f3f;
  bool bfs(){ queue <int> q;
    for(int i = base;i <= as;i++){ if(av[i] == -1) dist[i] = 0, q.push(i);
      else dist[i] = INF; }
    int mn = INF;
    while(!q.empty()){ int cur = q.front(); q.pop(); if(dist[cur] >= mn) continue;
      for(auto& nxt : adj[cur]){ int tmp = bv[nxt]; if(tmp == -1) mn = dist[cur] + 1;
        else if(dist[tmp] >= INF) dist[tmp] = dist[cur] + 1, q.push(tmp); } }
    return mn < INF; }
  bool dfs(int cur){ for(auto& nxt : adj[cur]){ int tmp = bv[nxt];
      if(tmp == -1 || (dist[tmp] == dist[cur] + 1 && dfs(tmp))){
        av[cur] = nxt; bv[nxt] = cur; return true; }
    }
    dist[cur] = INF; return false; }
public: _bm(int a = 0, int b = 0, bool base = 1){ clear(a, b, base); } // O(n)
  void clear(int a, int b, bool base = 1){ // O(n)
    as = a; bs = b; result = 0; this->base = base;
    adj.assign(as + 1, {}); dist.assign(as + 1, 0);
    av.assign(as + 1, -1); bv.assign(bs + 1, -1); }
  void addsol(int s, int e){ adj[s].push_back(e); } // O(1)
  int build(){ // O(m sqrt n)
    result = 0; fill(all(av), -1); fill(all(bv), -1); fill(all(dist), 0);
    while(bfs()){ for(int i = base;i <= as;i++){ if(av[i] != -1) continue;
        result += dfs(i); } }
    return result; }
  int ret() const{ return result; } // O(1)
  template <class F>
  void it_all(const F& f) const{ for(int i = base;i <= as;i++) f(i, av[i]); }
  template <class F>
  void it_matched(const F& f) const{
    for(int i = base;i <= as;i++) if(av[i] != -1) f(i, av[i]); } };
```

04 BCC / _bcc

```
class _bcc{ bool base; vector <ll> points;
  vector <pll> lines; vector <vector <pll>> blocks;
  void dfs(ll cur, ll pre = -1){ seq[cur] = mn[cur] = ++cnt;
    ll children = 0; bool cut = false, skipped_parent = false;
    for(auto nxt : adj[cur]){
      if(nxt == pre && !skipped_parent){ skipped_parent = true; continue; }
      if(seq[nxt]){ mn[cur] = min(mn[cur], seq[nxt]);
        if(seq[nxt] < seq[cur]) st.push({cur, nxt}); continue; }
      st.push({cur, nxt}); dfs(nxt, cur); children++; mn[cur] = min(mn[cur], mn[nxt]);
      if(mn[nxt] > seq[cur]) lines.push_back({min(cur, nxt), max(cur, nxt)});
      if(mn[nxt] >= seq[cur]){ if(pre != -1){ cut = true; num[cur]++; }
        vector <pll> block;
        while(true){ auto edge = st.top(); st.pop(); block.push_back(edge);
```

BCC / _bcc (계속)

```
          if(edge == pll(cur, nxt)) break; }
        blocks.push_back(move(block)); } }
    if(pre == -1 && children >= 2){ cut = true; num[cur] = children; }
    if(cut) points.push_back(cur); }
public: vector <vector <ll>> adj; vector <ll> mn, seq, num;
  ll n, cnt; stack <pll> st;
  _bcc(ll n = 0, bool base = 1){ clear(n, base); }
  void add(ll s, ll e){ adj[s].push_back(e); adj[e].push_back(s); }
  void clear(ll n, bool base = 1){ this->n = n; this->base = base;
    adj.assign(n + 1, {}); clear(); }
  void clear(){ seq.assign(n + 1, 0); mn.assign(n + 1, 0); num.assign(n + 1, 0); cnt = 0;
    while(!st.empty()) st.pop(); points.clear(); lines.clear(); blocks.clear(); }
  void build(){ // O(n + m)
    clear(); for(int i = base;i <= n;i++) if(!seq[i]) dfs(i); }
  const vector<ll>& point() const{ return points; } // O(1)
  const vector<pll>& line() const{ return lines; } // O(1)
  const vector<vector<pll>>& ret() const{ return blocks; } // O(1)
};
```

04 Hungarian / _hungarian

```
template <class C = ll, C CINF = INF>
class _hungarian{ bool base; vector <vector <C>> cost;
  vector <C> row, col; vector <int> match, pre; int n, sz;
  void loop(int i){ match[0] = i; int j0 = 0;
    vector <C> mn(sz + 1, CINF); vector <bool> use(sz + 1, 0);
    while(1){ use[j0] = 1; int i0 = match[j0], j1 = 0;
      C delta = CINF;
      for(int j = 1;j <= sz;j++){ if(use[j]) continue;
        C cur = cost[i0][j] - row[i0] - col[j];
        if(cur < mn[j]){ mn[j] = cur; pre[j] = j0;
        }
        if(mn[j] < delta){ delta = mn[j]; j1 = j;
        } }
      for(int j = 0;j <= sz;j++){ if(use[j]){ row[match[j]] += delta; col[j] -= delta; }
        else mn[j] -= delta; }
      j0 = j1; if(!match[j0]) break; }
    while(1){ int j1 = pre[j0]; match[j0] = match[j1];
      j0 = j1; if(!j0) break; }
  }
public: _hungarian(int n = 0, bool base = 1){ clear(n, base); }
  void clear(int n, bool base = 1){
    this->n = n; this->base = base; sz = n - base + 1; col.assign(sz + 1, 0);
    row.assign(sz + 1, 0); match.assign(sz + 1, 0);
    pre.assign(sz + 1, 0); cost.assign(sz + 1, vector<C>(sz + 1, 0)); }
  void set(int i, int j, C v){ assert(base <= i && i <= n && base <= j && j <= n);
    cost[i - base + 1][j - base + 1] = v; }
  C build(){
    fill(all(row), 0); fill(all(col), 0); fill(all(match), 0); fill(all(pre), 0);
    for(int i = 1;i <= sz;i++) loop(i); return -col[0]; }
};
```

05 Dinic / _flow

```
template <class T = ll> requires (is_integral_v<T> && is_signed_v<T>)
class _flow { bool base;
public: static consteval T make_inf() { T x = 0;
    for (size_t i = 0; i < sizeof(T); ++i) x = (x << 8) | T(0x3f); return x; }
```

Dinic / _flow (계속)

```
  static constexpr T INF = make_inf();
  struct handle { int cur, idx; };
  struct edge { int nxt, rev; T c, cap;
    edge(int nxt, int rev, T c) : nxt(nxt), rev(rev), c(c), cap(c) {} };
private: vector <int> d, w; int n; vector <vector<edge>> adj;
  bool bfs(int st, int en) { memset(d.data(), -1, sizeof(int) * (n + 1));
    queue <int> q; q.push(st); d[st] = 0;
    while (!q.empty()) { int cur = q.front(); q.pop();
      for (auto& e : adj[cur]) { if(e.c <= 0) continue; int nxt = e.nxt;
        if (d[nxt] != -1) continue; d[nxt] = d[cur] + 1; q.push(nxt); }
    }
    return d[en] != -1; }
  T dfs(int cur, int en, T val) { if (cur == en) return val;
    int &i = w[cur], sz = (int)adj[cur].size();
    for(;i < sz;++i){ edge &e = adj[cur][i]; if(e.c <= 0) continue;
      int nxt = e.nxt; if(d[nxt] != d[cur] + 1) continue;
      T mn = dfs(nxt, en, min(val, e.c)); if(mn <= 0) continue;
      e.c -= mn; adj[nxt][e.rev].c += mn; return mn; }
    return 0; }
public: _flow(int n = 0, bool base = 1){ clear(n, base); } // O(n)
  void clear(int n, bool base = 1){ // O(n)
    this->n = n; this->base = base; adj.assign(n + 1, {});
    d.assign(n + 1, -1); w.assign(n + 1, 0); }
  void reset(){ // O(n + m)
    for(int i = base;i <= n;i++){ for(auto& e : adj[i]) e.c = e.cap; }
  }
  handle addsol(int st, int en, T c = INF){  // O(1)
    assert(base <= st && st <= n && base <= en && en <= n);
    int re = (int)adj[en].size() + (st == en), rs = (int)adj[st].size();
    adj[st].push_back({en, re, c}); adj[en].push_back({st, rs, 0}); return {st, rs};
  }
  pair <handle, handle> add(int st, int en, T c = INF){ // O(1)
    auto h1 = addsol(st, en, c); auto h2 = addsol(en, st, c); return {h1, h2}; }
  T augment(int st, int en, T f = INF){ // O(n^2 * m)
    assert(base <= st && st <= n && base <= en && en <= n && st != en); T r = 0;
    while(bfs(st, en) && f){ memset(w.data(), 0, sizeof(int) * (n + 1));
      while(f){ T now = dfs(st, en, f); if(!now) break; r += now; f -= now;
      } }
    return r; }
  T build(int st, int en, T f = INF){ reset(); return augment(st, en, f); }
  bool reachable(int st, int en) const{ // O(m)
    vector <bool> v(n + 1, false); queue <int> q; v[st] = 1; q.push(st);
    while(!q.empty()){ int cur = q.front(); q.pop(); if(cur == en) return true;
      for(const auto& e : adj[cur]){ if(e.c <= 0 || v[e.nxt]) continue;
        v[e.nxt] = 1; q.push(e.nxt); } }
    return v[en]; }
  template <class F>
  void it_idx(int idx, const F& f) const{ // O(|adj[idx]|)
    for(auto& [nxt, rev, c, cap] : adj[idx]){ if(!cap) continue; f(idx, nxt, cap - c, c);
    } }
  template <class F>
  void it_all(const F& f) const{ // O(m)
    for(int i = base;i <= n;i++) it_idx(i, f); }
  edge& get_edge(const handle& h){ return adj[h.cur][h.idx]; } // O(1)
  const edge& get_edge(const handle& h) const{ return adj[h.cur][h.idx]; } // O(1)
  edge& get_rev(const handle& h){ auto& e = get_edge(h); return adj[e.nxt][e.rev]; } //
```

Dinic / _flow (계속)

```
↳   O(1)
  const edge& get_rev(const handle& h) const{ auto& e = get_edge(h); return
↳   adj[e.nxt][e.rev]; } // O(1)
  T get_flow(const handle& h) const{ auto& e = get_edge(h); return e.cap - e.c; } // O(1)
  void add_flow(const handle& h, T delta){ set_flow(h, get_flow(h) + delta); } // O(1)
  void set_flow(const handle& h, T f){ // O(1)
    auto& e = get_edge(h), &re = get_rev(h); e.c = e.cap - f; re.c = f; }
  T get_cap(const handle& h) const{ return get_edge(h).cap; } // O(1)
  void add_cap(const handle& h, T delta){ set_cap(h, get_cap(h) + delta); } // O(1)
  void set_cap(const handle& h, T cap){ // O(1)
    auto& e = get_edge(h), &re = get_rev(h); T f = e.cap - e.c; e.cap = cap;
    e.c = cap - f; re.c = f; } };
```

05 Min-Cost Max-Flow / _mcmf

```
template <class F = ll, class C = ll> requires (
  is_integral_v<F> && is_signed_v<F> && is_integral_v<C> && is_signed_v<C>
)
class _mcmf { bool base;
public:
  template <class T>
  static consteval T make_inf() { T x = 0;
    for (size_t i = 0; i < sizeof(T); ++i) x = (x << 8) | T(0x3f); return x; }
  static constexpr F FINF = make_inf<F>(); static constexpr C CINF = make_inf<C>();
  struct edge{ int nxt, rev; F cap, initial_cap; C cost;
    edge(int nxt, int rev, F cap, C cost) : nxt(nxt), rev(rev), cap(cap),
↳     initial_cap(cap), cost(cost) {}
  };
private: vector<vector<edge>> adj; vector<C> dist, pot;
  vector<int> work, lv; int n; bool use_spfa;
  void spfa(int st) { fill(all(pot), CINF); vector <bool> in(n + 1, 0); queue <int> q;
    pot[st] = 0; q.push(st); in[st] = 1;
    while (!q.empty()) { int cur = q.front(); q.pop(); in[cur] = 0;
      for(auto &e : adj[cur]){ if (e.cap <= 0) continue; int nxt = e.nxt;
        if (pot[nxt] <= pot[cur] + e.cost) continue; pot[nxt] = pot[cur] + e.cost;
        if (!in[nxt]) q.push(nxt), in[nxt] = 1; } }
    for(int i = base; i <= n; i++) if(pot[i] >= CINF) pot[i] = 0; }
  bool dijkstra(int st, int en){ using pci = pair<C, int>;
    fill(all(dist), CINF); dist[st] = 0;
    priority_queue<pci, vector<pci>, greater<pci>> pq; pq.push({0, st});
    while (!pq.empty()) { auto [cd, cur] = pq.top(); pq.pop();
      if (cd != dist[cur]) continue;
      for (int i = 0;i < (int)adj[cur].size();i++) { auto &e = adj[cur][i];
        if(e.cap <= 0) continue; int nxt = e.nxt;
        C nd = cd + e.cost + pot[cur] - pot[nxt]; if(nd >= dist[nxt]) continue;
        dist[nxt] = nd; pq.push({nd, nxt}); } }
    if (dist[en] >= CINF) return false;
    for (int i = base;i <= n;i++) { if (dist[i] < CINF) pot[i] += dist[i]; }
    return true; }
  bool bfs(int st, int en){ fill(all(lv), -1); queue <int> q; lv[st] = 0; q.push(st);
    while(!q.empty()){ int cur = q.front(); q.pop();
      for(auto& e : adj[cur]){ int nxt = e.nxt;
        if(e.cost + pot[cur] - pot[nxt] != 0) continue;
        if(lv[nxt] != -1 || e.cap <= 0) continue; lv[nxt] = lv[cur] + 1; q.push(nxt); }
    }
    return lv[en] != -1; }
  F dfs(int cur, int en, F f){ if(cur == en) return f;
```

Min-Cost Max-Flow / _mcmf (계속)

```
    for(int &i = work[cur]; i < (int)adj[cur].size(); i++){ auto &e = adj[cur][i];
      if(e.cap <= 0) continue; int nxt = e.nxt;
      if(e.cost + pot[cur] - pot[nxt] != 0) continue;
      if(lv[nxt] != lv[cur] + 1) continue; F now = dfs(nxt, en, min(f, e.cap));
      if(now <= 0) continue; e.cap -= now; adj[nxt][e.rev].cap += now; return now;
    }
    return 0; }
public:
  _mcmf(int n = 0, bool use_spfa = true, bool base = 1){ clear(n, use_spfa, base); } //
↳   O(n)
  void clear(int n, bool use_spfa = true, bool base = 1){ // O(n)
    this->n = n; this->base = base; this->use_spfa = use_spfa; adj.assign(n + 1, {});
    dist.assign(n + 1, CINF); pot.assign(n + 1, 0);
    work.assign(n + 1, 0); lv.assign(n + 1, -1); }
  void add(int st, int en, F cap, C cost){ // O(1)
    addsol(st, en, cap, cost); addsol(en, st, cap, cost); }
  void addsol(int st, int en, F cap, C cost){ // O(1)
    assert(base <= st && st <= n && base <= en && en <= n);
    int a = (int)adj[st].size(), b = (int)adj[en].size() + (st == en);
    adj[st].push_back({en, b, cap, cost}); adj[en].push_back({st, a, 0, -cost}); }
  pair<F, C> augment(int st, int en, F limit = FINF){
    assert(base <= st && st <= n && base <= en && en <= n && st != en);
    F flow = 0; C cost = 0; if(use_spfa) spfa(st);
    while(flow < limit && dijkstra(st, en)) { while(flow < limit && bfs(st, en)){
        fill(all(work), 0);
        while(flow < limit){ F now = dfs(st, en, limit - flow); if(now <= 0) break;
          flow += now; cost += now * (pot[en] - pot[st]); } }
    }
    return {flow, cost}; }
  void reset(){ for(auto& row : adj) for(auto& e : row) e.cap = e.initial_cap;
    fill(all(pot), 0); fill(all(dist), CINF); fill(all(work), 0); fill(all(lv), -1); }
  pair<F, C> build(int st, int en, F limit = FINF){
    reset(); return augment(st, en, limit); } };
```

06 Segment Tree · monoid / _seg

```
struct monoid{
  struct node { ll v;
    node(ll v) : v(v){} };
  static node id(){ return {0}; }
  static node leaf(){ return {0}; }
  static node op(const node& l, const node& r){ return { l.v + r.v
    }; } };
template <class policy = monoid>
class _seg { bool base;
public: using node = typename policy::node;
  node op(const node& l, const node& r) const{ return policy::op(l, r); }
  node id() const{ return policy::id(); }
private: int n, sz; vector <node> seg;
  node join(const node& a, const node& b, bool right) const{ return right ? op(a, b) :
↳   op(b, a); }
  void reset_storage(int n){ this->n = n; sz = 1; while(sz < n + 1) sz <<= 1;
    seg.assign(2 * sz, id()); }
  template<class F>
  int bisect(int p, bool right, const F& f) const{
    assert(right ? base <= p && p <= n + 1 : base - 1 <= p && p <= n); assert(f(id()));
    if(right) { if (p == n + 1) return n; } else { if (p == base - 1) return base; }
```

Segment Tree · monoid / _seg (계속)

```
    int i = right ? (p + sz) : (p + 1 + sz); node v = id();
    while(1){ if(!right) --i; if(right) while ((i & 1) == 0) i >>= 1;
      if(!right) while (i > 1 && (i & 1)) i >>= 1; node tmp1 = join(v, seg[i], right);
      if (!f(tmp1)) { while (i < sz) { i = (i << 1) | (!right);
          node tmp2 = join(v, seg[i], right);
          if(f(tmp2)){ v = tmp2; i += right ? 1 : -1; } }
        return right ? (i - sz - 1) : (i + 1 - sz); }
      v = tmp1; if(right) ++i; if((i & -i) == i) break; }
    return right ? n : base; }
public: _seg(int n = 0, bool base = 1){ clear(n, base); } // O(n)
  _seg(span<const node> arr, bool base = 1) : base(base){ make(arr); } // O(n)
  void clear(int n, bool base = 1){ // O(n)
    this->base = base; reset_storage(n);
    for(int i = base;i <= n;i++) seg[i + sz] = policy::leaf();
    for(int i = sz - 1;i >= 1;i--) seg[i] = op(seg[i << 1], seg[i << 1 | 1]); }
  void make(span<const node> arr){ // O(n)
    reset_storage(arr.empty() ? base - 1 : (int)arr.size() - 1);
    for(int i = base;i < (int)arr.size();i++) seg[i + sz] = arr[i];
    for(int i = sz - 1;i >= 1;i--) seg[i] = op(seg[i << 1], seg[i << 1 | 1]); }
  node query(int st, int en) const{ // O(log n)
    st = max((int)base, st); en = min(n, en); if(st > en) return id();
    node l = id(), r = id(); st += sz; en += sz;
    while(st <= en){ if(st & 1) l = op(l, seg[st++]); if(!(en & 1)) r = op(seg[en--], r);
      st >>= 1; en >>= 1; }
    return op(l, r); }
  void set(int idx, const node& v){ // O(log n)
    if(idx < base || idx > n) return; int p = idx + sz; seg[p] = v;
    for(p >>= 1; p; p >>= 1) seg[p] = op(seg[p << 1], seg[p << 1 | 1]); }
  template<class F>
  int max_right(int l, const F& f) const{ return bisect(l, 1, f); } // O(log n)
  template<class F>
  int min_left(int r, const F& f) const{  return bisect(r, 0, f); } // O(log n)
};
```

06 Lazy Segment Tree · action / _prop

```
struct action{
  struct node { ll v; int sz;
    node(ll v, int sz = 1) : v(v), sz(sz){} };
  struct lazy{ ll v;
    lazy(ll v) : v(v){}
    bool operator==(const lazy&) const = default; };
  static node id(){ return {0, 0}; }
  static node leaf(){ return {0, 1}; }
  static lazy lz_id(){ return {0}; }
  static node op(const node& l, const node& r){ return node{ l.v + r.v,
      l.sz + r.sz }; }
  static void apply(node& seg, const lazy& lz){ seg.v += lz.v * seg.sz; }
  static void compose(lazy& s, const lazy& p){ s.v += p.v; } };
template <class policy = action>
class _prop { bool base;
public: using node = typename policy::node; using lazy = typename policy::lazy;
  node op(const node& l, const node& r) const{ return policy::op(l, r); }
  node id() const{ return policy::id(); }
private: vector <node> seg; vector <lazy> lz; int sz, h, n;
  lazy lz_id() const{ return policy::lz_id(); }
  void prop(node& seg, const lazy& lz) const{ policy::apply(seg, lz); }
```

Lazy Segment Tree · action / _prop (계속)

```
  void comp(lazy& s, const lazy& p) const{ policy::compose(s, p); }
  node join(const node& a, const node& b, bool right) const{ return right ? op(a, b) :
↳   op(b, a); }
  void reset_storage(int n){ this->n = n; sz = 1; while(sz < n + 1) sz <<= 1;
    h = __lg(sz); seg.assign(2 * sz, id()); lz.assign(2 * sz, lz_id());
  }
  void push(int idx){ if(lz[idx] == lz_id()) return; apply(idx << 1, lz[idx]);
    apply(idx << 1 | 1, lz[idx]); lz[idx] = lz_id(); }
  void propagate(int idx){ for(int i = h;i > 0;i--){ int cur = idx >> i;
      push(cur); } }
  void apply(int idx, const lazy& now){ prop(seg[idx], now);
    if(idx < sz) comp(lz[idx], now); }
  void rebuild(int idx){ while(idx > 1){ idx >>= 1;
      seg[idx] = op(seg[idx << 1], seg[idx << 1 | 1]); if(lz[idx] == lz_id()) continue;
      prop(seg[idx], lz[idx]); } }
  template<class F>
  int bisect(int p, bool right, const F& f) {
    assert(right ? base <= p && p <= n + 1 : base - 1 <= p && p <= n); assert(f(id()));
    if(right) { if (p == n + 1) return n; } else { if (p == base - 1) return base; }
    int i = right ? (p + sz) : (p + 1 + sz);
    if(right) propagate(i); else propagate(i - 1); node v = id();
    while(1){ if(!right) --i; if(right) while ((i & 1) == 0) i >>= 1;
      if(!right) while (i > 1 && (i & 1)) i >>= 1; node tmp1 = join(v, seg[i], right);
      if (!f(tmp1)) { while (i < sz) { push(i);
          i = (i << 1) | (!right); node tmp2 = join(v, seg[i], right);
          if(f(tmp2)){ v = tmp2; i += right ? 1 : -1; } }
        return right ? (i - sz - 1) : (i + 1 - sz); }
      v = tmp1; if(right) ++i; if((i & -i) == i) break; }
    return right ? n : base; }
public: _prop(int n = 0, bool base = 1){ clear(n, base); } // O(n)
  _prop(span<const node> arr, bool base = 1) : base(base){ make(arr); } // O(n)
  void clear(int n, bool base = 1){ // O(n)
    this->base = base; reset_storage(n);
    for(int i = base;i <= n;i++) seg[i + sz] = policy::leaf();
    for(int i = sz - 1;i >= 1;i--) seg[i] = op(seg[i << 1], seg[i << 1 | 1]); }
  void make(span<const node> arr){ // O(n)
    reset_storage(arr.empty() ? base - 1 : (int)arr.size() - 1);
    for(int i = base;i < (int)arr.size();i++) seg[i + sz] = arr[i];
    for(int i = sz - 1;i >= 1;i--) seg[i] = op(seg[i << 1], seg[i << 1 | 1]); }
  node query(int idx){ // O(log n)
    assert(base <= idx && idx <= n); idx += sz; propagate(idx); return seg[idx];
  }
  node query(int st, int en) { // O(log n)
    st = max((int)base, st); en = min(n, en); if(st > en) return id();
    int l = (int)st + sz, r = (int)en + sz; propagate(l); propagate(r);
    node nl = id(), nr = id();
    while(l <= r){ if(l & 1) nl = op(nl, seg[l++]); if(!(r & 1)) nr = op(seg[r--], nr);
      l >>= 1; r >>= 1; }
    return op(nl, nr); }
  void update(int idx, const lazy& lz){ update(idx, idx, lz); } // O(log n)
  void update(int st, int en, const lazy& lz){ // O(log n)
    st = max((int)base, st); en = min(n, en); if(st > en) return;
    int l = (int)st + sz, r = (int)en + sz; int tl = l, tr = r;
    propagate(l); propagate(r);
    while(l <= r){ if(l & 1) apply(l++, lz); if(!(r & 1)) apply(r--, lz);
      l >>= 1; r >>= 1; }
```

Lazy Segment Tree · action / _prop (계속)

```
    rebuild(tl); rebuild(tr); }
  template<class F>
  int max_right(int l, const F& f){ return bisect(l, 1, f); } // O(log n)
  template<class F>
  int min_left(int r, const F& f){  return bisect(r, 0, f); } // O(log n)
};
```

06 HLD / _hld

```
class _hld { protected: bool base;
  vector<vector<int>> adj, tmp; vector<int> sz, top, d, in, out, p, at;
  int n, m, cnt = 0;
private: void build_tree(int cur, int pre = -1){ for(auto& nxt : tmp[cur]){
      if(pre == nxt) continue; adj[cur].push_back(nxt); build_tree(nxt, cur);
    } }
  void dfs1(int cur){ sz[cur] = 1;
    for(auto& nxt : adj[cur]){ d[nxt] = d[cur] + 1; p[nxt] = cur;
      dfs1(nxt); sz[cur] += sz[nxt];
      if(sz[nxt] > sz[adj[cur][0]]) swap(nxt, adj[cur][0]); } }
  void dfs2(int cur){ in[cur] = cnt++; at[in[cur]] = cur;
    for(auto& nxt : adj[cur]){ top[nxt] = (nxt == adj[cur][0] ? top[cur] : nxt);
      dfs2(nxt); }
    out[cur] = cnt - 1; }
public: _hld(int n = 0, int m = 0, bool base = 1){ clear(n, m, base); } // O(n)
  void clear(int n, int m = 0, bool base = 1){ // O(n)
    this->n = n; this->base = base; this->m = m; cnt = 0;
    adj.assign(n + 1, {}); tmp.assign(n + 1, {});
    top.assign(n + 1, -1); d.assign(n + 1, 0); in.assign(n + 1, 0); out.assign(n + 1, 0);
    p.assign(n + 1, -1); sz.assign(n + 1, 0); at.assign(n + 1, -1); }
  void addsol(int s, int e){ tmp[s].push_back(e); } // O(1)
  void add(int s, int e){ addsol(s, e); addsol(e, s); } // O(1)
  void build(){ // O(n)
    cnt = 0; adj.assign(n + 1, {});
    fill(all(sz), 0); fill(all(top), -1); fill(all(d), 0);
    fill(all(in), 0); fill(all(out), 0); fill(all(p), -1); fill(all(at), -1);
    for(int i = base; i <= n; i++){ if(sz[i]) continue;
      top[i] = i; build_tree(i); dfs1(i); dfs2(i); } }
  int index(int v) const{ return in[v]; } // O(1)
  int vertex(int i) const{ return at[i]; } // O(1)
  int parent(int v) const{ return p[v]; } // O(1)
  int depth(int v) const{ return d[v]; } // O(1)
  pair<int,int> subtree(int v) const{ return {in[v] + m, out[v]}; } // O(1)
  template<class F>
  void it_path(int st, int en, const F& f) const{ // O(log n)
    vector<pair<int,int>> tail;
    while(top[st] != top[en]){ if(d[top[st]] >= d[top[en]]){
        int cur = top[st], l = in[cur], r = in[st]; if(m == 1 && p[cur] == -1) l++;
        if(l <= r) f(l, r, 1); st = p[cur]; }
      else{ int cur = top[en], l = in[cur], r = in[en]; if(m == 1 && p[cur] == -1) l++;
        if(l <= r) tail.push_back({l, r}); en = p[cur]; }
    }
    bool rev = d[st] > d[en]; int l = in[rev ? en : st] + m, r = in[rev ? st : en];
    if(l <= r) f(l, r, rev);
    for(auto it = tail.rbegin();it != tail.rend();++it) f(it->first, it->second, 0); } };
```

06 HLD 경로 연산 / _hld_path

_hld + backend(_seg 또는 _prop) 필요. 간선 모드 m=1.

```
template<class P>
concept is_action = requires(P s, int l, int r, const typename P::lazy& z){ s.update(l, r, z); };
template<class backend>
class _hld_path : public _hld { public: using node = typename backend::node;
  backend seg;
private:
  static constexpr bool can_const_query = requires(const backend& s, int l, int r, const node& a){
    { s.query(l, r) } -> std::convertible_to<node>;
    { s.id() } -> std::convertible_to<node>; { s.op(a, a) } -> std::convertible_to<node>;
  }; bool built = 0;
  void reset_seg(){
    if constexpr (requires(backend& s){ s.clear(n, 0); }) seg.clear(n, 0);
    else seg.clear(n); }
  template<class S>
  node query_path(S& s, int st, int en) const{ node ret = s.id();
    it_path(st, en, [&](int l, int r, bool rev){ node part = s.query(l, r);
      if constexpr (requires(const node& v){ { v.reversed() } -> std::same_as<node>; }){
        if(rev) part = part.reversed(); }
      ret = s.op(ret, part); }); return ret;
  }
public: _hld_path(int n = 0, int m = 0, bool base = 1) : _hld(n, m, base){ reset_seg(); }
  void clear(int n, int m = 0, bool base = 1){
    _hld::clear(n, m, base); reset_seg(); built = 0; }
  void build(){ vector<node> values;
    if(built){ values.assign(n + 1, seg.id());
      for(int v = base; v <= n; ++v) values[v] = seg.query(in[v], in[v]); }
    _hld::build(); if(built) make(values); built = 1;
  }
  void make(span<const node> values){ vector<node> ordered(n + 1, seg.id());
    for(int v = base; v <= n && v < (int)values.size(); ++v) ordered[in[v]] = values[v];
    if constexpr (requires(backend& s, span<const node> a){ s.make(a); })
      seg.make(ordered);
    else { reset_seg(); for(int v = base; v <= n; ++v) seg.set(in[v], ordered[in[v]]);
    } }
  node query(int x){ return seg.query(in[x], in[x]); }
  node query(int x) const requires can_const_query{ return seg.query(in[x], in[x]); }
  node query(int st, int en){ // O(log^2 n)
    return query_path(seg, st, en); }
  node query(int st, int en) const requires can_const_query{ // O(log^2 n)
    return query_path(seg, st, en); }
  node query_sub(int x){ auto [l, r] = subtree(x); return seg.query(l, r); }
  node query_sub(int x) const requires can_const_query{ auto [l, r] = subtree(x); return seg.query(l, r); }
  void set(int x, const node& v)
    requires requires(backend s, int i, const node& nv){ s.set(i, nv); }
  { seg.set(in[x], v); }
  void update(int x, const node& v)
    requires requires(backend s, int i, const node& nv){ s.update(i, nv); }
  { seg.update(in[x], v); }
  template<class P = backend>
  void update(int st, int en, const typename P::lazy& lz)
    requires is_action<P>
  { it_path(st, en, [&](int l, int r, bool){ seg.update(l, r, lz); }); }
```



```
  template<class P = backend>
  void update_sub(int x, const typename P::lazy& lz)
    requires is_action<P>
  { auto [l, r] = subtree(x); seg.update(l, r, lz); } };
```

06 Merge Sort Tree / _mt

```
class mt_policy{ public:
  class node{ public: ll v;
    node(ll v) : v(v){}
    bool operator < (const node& ot) const{ return v < ot.v; } }; };
template <class policy = mt_policy>
class _mt{ bool base;
public: using node = typename policy::node;
private: vector <vector <node>> seg; int n, sz = 1;
  static int count_node(const vector<node>& cur, const node& v, bool is_greater, bool is_less, bool is_eq){
    int ret = 0; auto lb = lower_bound(all(cur), v); auto ub = upper_bound(all(cur), v);
    if(is_eq) ret += (int)(ub - lb); if(is_greater) ret += (int)(cur.end() - ub);
    if(is_less) ret += (int)(lb - cur.begin()); return ret; }
  int query(int st, int en, const node& v, bool is_greater, bool is_less, bool is_eq)
    const{
    st = max(st, (int)base); en = min(en, n); if(n < base || st > en) return 0;
    int l = sz + st, r = sz + en, ret = 0;
    while(l <= r){ if(l & 1) ret += count_node(seg[l++], v, is_greater, is_less, is_eq);
      if(!(r & 1)) ret += count_node(seg[r--], v, is_greater, is_less, is_eq);
      l >>= 1; r >>= 1; }
    return ret; }
public: _mt(int n = 0, bool base = 1){ clear(n, base); } // O(n)
  _mt(span<const node> arr, bool base = 1) : base(base){ make(arr); } // O(n log n)
  void clear(int n, bool base = 1){ // O(n)
    this->n = n; this->base = base; sz = 1; while(sz < n + 1) sz <<= 1;
    seg.assign(2 * sz, {}); }
  void make(span<const node> arr){ // O(n log n)
    if(arr.empty()){ clear(base - 1, base); return; }
    clear((int)arr.size() - 1, base);
    for(int i = base; i <= n; i++) seg[sz + i].assign(1, arr[i]);
    for(int i = sz - 1; i >= 1; i--){
      auto &l = seg[i << 1], &r = seg[i << 1 | 1], &cur = seg[i];
      cur.clear(); cur.reserve(l.size() + r.size());
      merge(all(l), all(r), back_inserter(cur)); } }
  void update(int idx, const node& v){ // O(n)
    if(idx < base || idx > n) return; int p = sz + idx; seg[p].assign(1, v);
    for(int i = p >> 1;i >= 1;i >>= 1){
      auto &l = seg[i << 1], &r = seg[i << 1 | 1], &cur = seg[i];
      cur.clear(); cur.reserve(l.size() + r.size());
      merge(all(l), all(r), back_inserter(cur)); } }
  int greater(int st, int en, const node& v) const{ return query(st, en, v, 1, 0, 0); }
  // O(log^2 n)
  int less(int st, int en, const node& v) const{ return query(st, en, v, 0, 1, 0); } //
  O(log^2 n)
  int geq(int st, int en, const node& v) const{ return query(st, en, v, 1, 0, 1); } //
  O(log^2 n)
  int leq(int st, int en, const node& v) const{ return query(st, en, v, 0, 1, 1); } //
  O(log^2 n)
  int eq(int st, int en, const node& v) const{ return query(st, en, v, 0, 0, 1); } //
  O(log^2 n)
```

Merge Sort Tree / _mt (계속)

```
};
```

06 가환 group · has_inv

_fw, _fw2d, _pst_diff 에서 사용.

```
template <class policy>
concept has_inv = requires(const typename policy::node& a){
  { policy::inv(a) } -> std::same_as<typename policy::node>; };
struct group{
  struct node{ ll v;
    node(ll v) : v(v){} };
  static node id(){ return {0}; }
  static node leaf(){ return {0}; }
  static node op(const node& l, const node& r){ return {l.v + r.v}; }
  static node inv(const node& a){ return {-a.v}; }
};
```

06 Persistent Segment Tree / _pst

monoid 필요.

```
template <class policy = monoid>
class _pst{ protected: bool base;
public: using node = typename policy::node;
  node op(const node& l, const node& r) const{ return policy::op(l, r); }
  node id() const{ return policy::id(); }
protected: vector <int> lv, rv; vector <node> seg;
  ll n;
private: int clone(int idx){ int l = lv[idx], r = rv[idx];
    node v = seg[idx]; lv.push_back(l); rv.push_back(r); seg.push_back(v);
    return (int)seg.size() - 1; }
  int set(int idx, ll s, ll e, ll pos, const node& v){ int cur = clone(idx);
    if(s == e){ seg[cur] = v; return cur; }
    ll m = midpoint(s, e); if(pos <= m) lv[cur] = set(lv[cur], s, m, pos, v);
    else rv[cur] = set(rv[cur], m + 1, e, pos, v);
    seg[cur] = op(seg[lv[cur]], seg[rv[cur]]); return cur; }
  node query(int idx, ll s, ll e, ll l, ll r) const{
    if(!idx || r < s || e < l) return id(); if(l <= s && e <= r) return seg[idx];
    ll m = midpoint(s, e); node ln = query(lv[idx], s, m, l, r);
    node rn = query(rv[idx], m + 1, e, l, r); return op(ln, rn); }
  template<class F>
  ll bisect(int idx, ll s, ll e, ll p, bool right, const F& f, node& v) const{
    if(right && e < p) return p - 1; if(!right && p < s) return p + 1;
    if(!idx) return right ? e : s;
    if((right && p <= s) || (!right && e <= p)){
      node nxt = right ? op(v, seg[idx]) : op(seg[idx], v);
      if(f(nxt)){ v = nxt; return right ? e : s; }
      if(s == e) return right ? s - 1 : s + 1; }
    ll m = midpoint(s, e);
    if(right){ ll ret = bisect(lv[idx], s, m, p, right, f, v); if(ret < m) return ret;
      return bisect(rv[idx], m + 1, e, p, right, f, v); }
    ll ret = bisect(rv[idx], m + 1, e, p, right, f, v); if(ret > m + 1) return ret;
    return bisect(lv[idx], s, m, p, right, f, v); }
public: _pst(ll n = 0, bool base = 1){ clear(n, base); }
  void clear(ll n, bool base = 1){ this->n = n; this->base = base;
    lv.assign(1, 0); rv.assign(1, 0); seg.assign(1, id()); }
  int node_count() const{ return (int)seg.size() - 1; }
  int set(int rt, ll idx, const node& v){ // O(log n)
    assert(base <= idx && idx <= n); return set(rt, base, n, idx, v); }
```

Persistent Segment Tree / _pst (계속)

```
  node get(int rt, ll idx) const{ // O(log n)
    return query(rt, idx, idx); }
  node query(int rt, ll l, ll r) const{ // O(log n)
    l = max((ll)base, l); r = min(n, r); if(l > r) return id();
    return query(rt, base, n, l, r); }
  template<class F>
  ll max_right(int rt, ll l, const F& f) const{
    assert(base <= l && l <= n + 1); assert(f(id())); if(l > n) return n; node v = id();
    return bisect(rt, base, n, l, 1, f, v); }
  template<class F>
  ll min_left(int rt, ll r, const F& f) const{
    assert(base - 1 <= r && r <= n); assert(f(id())); if(r < base) return base;
    node v = id(); return bisect(rt, base, n, r, 0, f, v); }
};
```

06 PST 버전 차이 / _pst_diff

_pst + group + has_inv 필요.

```
template<class policy = group> requires has_inv<policy>
class _pst_diff : public _pst<policy> { private: using tree = _pst<policy>;
  using tree::base; using tree::n; using tree::lv;
  using tree::rv; using tree::seg;
public: using node = typename tree::node; using tree::op;
  using tree::id;
  node inv(const node& a) const{ return policy::inv(a); }
private: node diff_seg(int idx1, int idx2) const{ return op(seg[idx2], inv(seg[idx1])); }
  }
  template<class F>
  ll bisect_diff(int idx1, int idx2, ll s, ll e, ll p, bool right, const F& f, node& v)
↳   const{
    if(right && e < p) return p - 1; if(!right && p < s) return p + 1;
    if(!idx1 && !idx2) return right ? e : s;
    if((right && p <= s) || (!right && e <= p)){ node diff = diff_seg(idx1, idx2);
      node nxt = right ? op(v, diff) : op(diff, v);
      if(f(nxt)){ v = nxt; return right ? e : s; }
      if(s == e) return right ? s - 1 : s + 1; }
    ll m = midpoint(s, e);
    if(right){ ll ret = bisect_diff(lv[idx1], lv[idx2], s, m, p, right, f, v);
      if(ret < m) return ret;
      return bisect_diff(rv[idx1], rv[idx2], m + 1, e, p, right, f, v); }
    ll ret = bisect_diff(rv[idx1], rv[idx2], m + 1, e, p, right, f, v);
    if(ret > m + 1) return ret;
    return bisect_diff(lv[idx1], lv[idx2], s, m, p, right, f, v); }
public: _pst_diff(ll n = 0, bool base = 1) : tree(n, base){}
  node diff_get(int rt1, int rt2, ll idx) const{ return diff_query(rt1, rt2, idx, idx); }
  node diff_query(int rt1, int rt2, ll l, ll r) const{
    l = max((ll)base, l); r = min(n, r); if(l > r) return id();
    return op(this->query(rt2, l, r), inv(this->query(rt1, l, r))); }
  template<class F>
  ll diff_max_right(int rt1, int rt2, ll l, const F& f) const{
    assert(base <= l && l <= n + 1); assert(f(id())); if(l > n) return n; node v = id();
    return bisect_diff(rt1, rt2, base, n, l, 1, f, v); }
  template<class F>
  ll diff_min_left(int rt1, int rt2, ll r, const F& f) const{
    assert(base - 1 <= r && r <= n); assert(f(id())); if(r < base) return base;
    node v = id(); return bisect_diff(rt1, rt2, base, n, r, 0, f, v); }
};
```

06 Fenwick Tree / _fw

group + has_inv 필요.

```
template <class policy = group>
class _fw{ bool base;
public: using node = typename policy::node;
  node op(const node& l, const node& r) const{ return policy::op(l, r); }
  node id() const{ return policy::id(); }
private: vector <node> bit; int n;
  node inv(const node& a) const{ return policy::inv(a); }
public: _fw(int n = 0, bool base = 1){ clear(n, base); } // O(n)
  _fw(span<const node> arr, bool base = 1) : base(base){ make(arr); } // O(n)
  void clear(int n, bool base = 1){ // O(n)
    this->n = n; this->base = base; bit.assign(n + 2, id()); }
  void make(span<const node> arr){ // O(n)
    if(arr.empty()){ clear(base - 1, base); return; }
    clear((int)arr.size() - 1, base); for(int i = base;i <= n;i++) bit[i + 1] = arr[i];
    for(int i = 1;i <= n + 1;i++){ int j = i + (i & -i);
      if(j <= n + 1) bit[j] = op(bit[j], bit[i]); } }
  // range query -> need inv
  node query(int idx) const requires has_inv<policy> { return query(idx, idx); } //
↳   O(log n)
  node query(int l, int r) const requires has_inv<policy> { // O(log n)
    l = max(l, (int)base); r = min(r, n); if(l > r) return id();
    return op(pre(r), inv(pre(l - 1))); }
  // return 0 ~ idx
  node pre(int idx) const{ // O(log n)
    node ret = id(); if(idx < base) return ret; idx = min(idx, n);
    for(int i = idx + 1;i > 0;i -= i & -i) ret = op(ret, bit[i]); return ret; }
  void set(int idx, const node& v) requires has_inv<policy> { // O(log n)
    if(idx < base || idx > n) return; node cur = query(idx, idx), d = op(v, inv(cur));
    update(idx, d); }
  void update(int idx, const node& v){ // O(log n)
    if(idx < base || idx > n) return;
    for(int i = idx + 1;i <= n + 1;i += i & -i) bit[i] = op(bit[i], v); } };
```

06 2D Fenwick Tree / _fw2d

group + has_inv 필요.

```
template <class policy = group>
class _fw2d{ bool base;
public: using node = typename policy::node;
  node op(const node& l, const node& r) const { return policy::op(l, r); }
  node id() const { return policy::id(); }
private: vector<vector<node>> bit; int n, m;
  node inv(const node& a) const { return policy::inv(a); }
public: _fw2d(int n = 0, int m = 0, bool base = 1){ clear(n, m, base); } // O(nm)
  void clear(int n, int m, bool base = 1){ // O(nm)
    this->n = n; this->base = base; this->m = m;
    bit.assign(n + 2, vector<node>(m + 2, id())); }
  // return (0,0) ~ (y,x)
  node pre(int y, int x) const{ // O(log n log m)
    if(y < base || x < base) return id(); node ret = id(); x = min(x, m); y = min(y, n);
    for(int i = y + 1; i > 0; i -= i & -i){ for(int j = x + 1; j > 0; j -= j & -j){
        ret = op(ret, bit[i][j]); } }
    return ret; }
  // range query -> need inv
  node query(int y, int x) const requires has_inv<policy> { return query(y, x, y, x); }
↳   // O(log n log m)
```

2D Fenwick Tree / _fw2d (계속)

```
  node query(int y1, int x1, int y2, int x2) const requires has_inv<policy> { // O(log n
↳   log m)
    x1 = max(x1, (int)base); y1 = max(y1, (int)base); x2 = min(x2, m); y2 = min(y2, n);
    if(x1 > x2 || y1 > y2) return id(); node a = pre(y2, x2), b = pre(y2, x1 - 1);
    node c = pre(y1 - 1, x2), d = pre(y1 - 1, x1 - 1);
    node ab = op(a, inv(b)), cd = op(inv(c), d); return op(ab, cd); }
  void update(int y, int x, const node& v){ // O(log n log m)
    if(y < base || y > n || x < base || x > m) return;
    for(int i = y + 1; i <= n + 1; i += i & -i){
      for(int j = x + 1; j <= m + 1; j += j & -j){ bit[i][j] = op(bit[i][j], v); }
    } }
  void set(int y, int x, const node& v) requires has_inv<policy> { // O(log n log m)
    if(y < base || y > n || x < base || x > m) return;
    node cur = query(y, x), d = op(v, inv(cur)); update(y, x, d); }
};
```

06 정·역방향 **policy** 어댑터

bidirectional / bidirectional_action: 비가환 경로 집계용.

```
template <class P>
class bidirectional{ public: using value = typename P::node;
  class node{ public: value forward, backward;
    node(value forward, value backward)
      : forward(std::move(forward)), backward(std::move(backward)){}
    node reversed() const{ return {backward, forward}; } };
  static node lift(const value& v){ return {v, v}; }
  static node id(){ return lift(P::id()); }
  static node leaf(){ return lift(P::leaf()); }
  static node op(const node& l, const node& r){
    return {P::op(l.forward, r.forward), P::op(r.backward, l.backward)}; } };
template <class P>
class bidirectional_action : public bidirectional<P>{ public:
  using node = typename bidirectional<P>::node; using lazy = typename P::lazy;
  static lazy lz_id(){ return P::lz_id(); }
  static void apply(node& value, const lazy& tag){
    P::apply(value.forward, tag); P::apply(value.backward, tag); }
  static void compose(lazy& old, const lazy& next){ P::compose(old, next); } };
```

07 문자 집합 / **alphabet**

```
namespace alphabet{
  class lowercase{ public: static constexpr int size = 26;
    static constexpr int encode(unsigned char c){ return c - 'a'; }
    static constexpr unsigned char decode(int i){ return 'a' + i; }
  };
  class uppercase{ public: static constexpr int size = 26;
    static constexpr int encode(unsigned char c){ return c - 'A'; }
    static constexpr unsigned char decode(int i){ return 'A' + i; }
  };
  class digits{ public: static constexpr int size = 10;
    static constexpr int encode(unsigned char c){ return c - '0'; }
    static constexpr unsigned char decode(int i){ return '0' + i; }
  };
  class alnum{ public: static constexpr int size = 62;
    static constexpr int encode(unsigned char c){
      if('0' <= c && c <= '9') return c - '0';
      if('A' <= c && c <= 'Z') return 10 + c - 'A'; return 36 + c - 'a'; }
    static constexpr unsigned char decode(int i){ if(i < 10) return '0' + i;
```

문자 집합 / alphabet (계속)

```
      if(i < 36) return 'A' + i - 10; return 'a' + i - 36; }
  };
  class bytes{ public: static constexpr int size = 256;
    static constexpr int encode(unsigned char c){ return c; }
    static constexpr unsigned char decode(int i){ return static_cast<unsigned char>(i); }
  };
  template <class Alphabet, class T = int>
  vector <T> tf(const string& s){ // O(|s|)
    vector <T> ret; ret.reserve(s.size());
    for(unsigned char c : s) ret.push_back(static_cast<T>(Alphabet::encode(c)));
    return ret; } }
```

07 Trie / _trie

alphabet 필요. 기본 문자 집합은 소문자 .

```
template <class Alphabet = alphabet::lowercase>
class _trie { // 0-based index
private: vector <array<int, Alphabet::size>> adj; vector <int> cnt, pass;
  int new_node(){
    adj.push_back(array<int, Alphabet::size>{}); cnt.push_back(0); pass.push_back(0);
    return (int)adj.size() - 1; }
public:
  class state{ private: friend class _trie;
    const _trie* owner; int idx;
    state(const _trie* owner, int idx) : owner(owner), idx(idx){}
  public: int id() const{ return idx; } // O(1)
    int count() const{ return idx == -1 ? 0 : owner->cnt[idx]; } // O(1)
    int prefix_count() const{ return idx == -1 ? 0 : owner->pass[idx]; } // O(1)
    bool end() const{ return count() > 0; } // O(1)
    bool valid() const{ return id() != -1; } // O(1)
    bool has_child() const{ return prefix_count() > count(); } // O(1)
    state go(int c) const{ // O(1)
      if(idx < 0 || idx >= (int)owner->adj.size()) return {owner, -1};
      int nxt = owner->adj[idx][c]; return {owner, nxt ? nxt : -1}; }
    template<class F>
    void it_child(const F& f) const{ // O(Alphabet::size)
      if(idx < 0 || idx >= (int)owner->adj.size()) return;
      for(int c = 0;c < Alphabet::size;c++){ int nxt = owner->adj[idx][c];
        if(nxt && owner->pass[nxt]) f(c, state(owner, nxt)); } }
    template<class F>
    void it_subtree(const F& f) const{ // O(k * Alphabet::size)
      if(idx < 0 || idx >= (int)owner->adj.size() || !owner->pass[idx]) return; f(*this);
      it_child([&](int, state nxt){ nxt.it_subtree(f); }); } };
  _trie(){ clear(); } // O(1)
  void clear(){ adj.clear(); cnt.clear(); pass.clear(); new_node(); } // O(1)
  state get_state(const string& s) const{ auto v = alphabet::tf<Alphabet>(s); return
↳   get_state(v); } // O(|s|)
  state get_state(span<const int> v) const{ // O(|v|)
    int cur = 0;
    for(auto& c : v){ if (!adj[cur][c]) return {this, -1}; cur = adj[cur][c];
    }
    return {this, cur}; }
  int size() const{ return pass.empty() ? 0 : pass[0]; } // O(1)
  state insert(const string& s){ auto v = alphabet::tf<Alphabet>(s); return insert(v); }
↳   // O(|s|)
  state insert(span<const int> v){ // O(|v|)
    int cur = 0; pass[cur]++;
```

Trie / _trie (계속)

```
    for(auto &c : v){ if(!adj[cur][c]) adj[cur][c] = new_node();
      cur = adj[cur][c]; pass[cur]++; }
    cnt[cur]++; return {this, cur}; }
  bool erase(const string& s){ auto v = alphabet::tf<Alphabet>(s); return erase(v); } //
↳   O(|s|)
  bool erase(span<const int> v){ // O(|v|)
    int state = get_state(v).id(); if(state == -1 || !cnt[state]) return 0;
    int cur = 0; cnt[state]--; pass[cur]--;
    for(auto &c : v){ cur = adj[cur][c]; pass[cur]--; }
    return 1; }
  int rank(const string& s) const{ auto v = alphabet::tf<Alphabet>(s); return rank(v); }
↳   // O(|s| * Alphabet::size)
  int rank(span<const int> v) const{ // O(|v| * Alphabet::size)
    int ret = 0, cur = 0;
    for(auto& c : v){ ret += cnt[cur];
      for(int x = 0;x < c;x++){ int nxt = adj[cur][x]; if(nxt) ret += pass[nxt];
      }
      int nxt = adj[cur][c]; if(!nxt) return ret; cur = nxt; }
    return ret; }
  vector <int> kth(int k) const{ // O(max_len * Alphabet::size)
    if(k <= 0 || k > pass[0]) return {}; vector <int> ret; int cur = 0;
    while(1){ if(k <= cnt[cur]) break; k -= cnt[cur];
      for(int c = 0;c < Alphabet::size;c++){ int nxt = adj[cur][c]; if(!nxt) continue;
        if(pass[nxt] >= k){ ret.push_back(c); cur = nxt; break;
        } else k -= pass[nxt]; }
    }
    return ret; } };
```

07 XOR Trie / _xor_trie

```
class _xor_trie { // 0-based index
private: vector <array<int, 2>> adj; vector <int> pass; int m; u64 tag;
  int new_node(){ adj.push_back({0, 0}); pass.push_back(0); return (int)adj.size() - 1;
  }
  u64 arg(u64 x, bool is_max) const{ if(!size()) return 0; int cur = 0; u64 ret = 0;
    x ^= tag;
    for(int i = m;i >= 0;i--){ bool bit = (x >> i) & 1ull, need = bit ^ is_max;
      int nxt = adj[cur][need];
      if(nxt && pass[nxt] > 0){ ret |= (u64)(need) << i; cur = nxt; }
      else{ ret |= (u64)(need ^ 1) << i; cur = adj[cur][need ^ 1];
      } }
    return ret ^ tag; }
public:
  class state{ private: friend class _xor_trie;
    const _xor_trie* owner; int idx, bit;
    state(const _xor_trie* owner, int idx, int bit) : owner(owner), idx(idx), bit(bit){}
  public: int id() const{ return idx; } // O(1)
    int count() const{ return idx == -1 || bit >= 0 ? 0 : owner->pass[idx]; } // O(1)
    int prefix_count() const{ return idx == -1 ? 0 : owner->pass[idx]; } // O(1)
    bool end() const{ return count() > 0; } // O(1)
    bool valid() const{ return id() != -1; } // O(1)
    bool has_child() const{ return prefix_count() > count(); } // O(1)
    state go(bool c) const{ // O(1)
      if(idx < 0 || idx >= (int)owner->adj.size() || bit < 0) return {owner, -1, -1};
      bool flip = (owner->tag >> bit) & 1ull; int nxt = owner->adj[idx][c ^ flip];
      return {owner, nxt ? nxt : -1, bit - 1}; }
    template<class F>
```

XOR Trie / _xor_trie (계속)

```
    void it_child(const F& f) const{ // O(1)
      if(idx < 0 || idx >= (int)owner->adj.size() || bit < 0) return;
      bool flip = (owner->tag >> bit) & 1ull;
      for(int c = 0;c < 2;c++){ int nxt = owner->adj[idx][c ^ flip];
        if(nxt && owner->pass[nxt]) f(c, state(owner, nxt, bit - 1)); } }
    template<class F>
    void it_subtree(const F& f) const{ // O(k)
      if(idx < 0 || idx >= (int)owner->adj.size() || !owner->pass[idx]) return; f(*this);
      it_child([&](int, state nxt){ nxt.it_subtree(f); }); } };
  _xor_trie(int max_bit = 63){ clear(max_bit); } // O(1)
  void clear(int max_bit){ // O(1)
    this->m = max_bit; tag = 0; adj.clear(); pass.clear(); new_node(); }
  void xor_all(u64 mask){ tag ^= mask; } // O(1)
  state root() const{ return {this, 0, m}; } // O(1)
  state get_state(u64 x) const{ // O(max_bit)
    x ^= tag; int cur = 0;
    for(int i = m;i >= 0;i--){ bool bit = (x >> i) & 1ull; int nxt = adj[cur][bit];
      if(!nxt) return {this, -1, -1}; cur = nxt; }
    return {this, cur, -1}; }
  int size() const { return pass.empty() ? 0 : pass[0]; } // O(1)
  state insert(u64 x){ // O(max_bit)
    x ^= tag; int cur = 0; pass[cur]++;
    for(int i = m;i >= 0;i--){ bool bit = (x >> i) & 1ull;
      if(!adj[cur][bit]) adj[cur][bit] = new_node(); cur = adj[cur][bit]; pass[cur]++; }
    return {this, cur, -1}; }
  bool erase(u64 x){ // O(max_bit)
    int state = get_state(x).id(); if(state == -1 || !pass[state]) return 0; x ^= tag;
    int cur = 0; pass[cur]--;
    for(int i = m;i >= 0;i--){ bool bit = (x >> i) & 1ull;
      cur = adj[cur][bit]; pass[cur]--; }
    return 1; }
  u64 max(u64 x) const{ return x ^ argmax(x); } // O(max_bit)
  u64 argmax(u64 x) const{ return arg(x, 1); } // O(max_bit)
  u64 min(u64 x) const{ return x ^ argmin(x); } // O(max_bit)
  u64 argmin(u64 x) const{ return arg(x, 0); } // O(max_bit)
  u64 kth(u64 x, int k) const{ return x ^ argkth(x, k); } // O(max_bit + 1)
  u64 argkth(u64 x, int k) const{ // O(max_bit + 1)
    x ^= tag; int cur = 0; u64 ret = 0;
    for(int i = m;i >= 0;i--){ bool bit = (x >> i) & 1ull;
      int nxt = adj[cur][bit], num = nxt ? pass[nxt] : 0;
      if(k > num){ k -= num; bit = !bit; }
      cur = adj[cur][bit]; ret |= (u64)bit << i; }
    return ret ^ tag; }
  int greater(u64 x, u64 k) const{ return k == numeric_limits<u64>::max() ? 0 : size() -
↳   less(x, k + 1); } // O(max_bit)
  int equal(u64 x, u64 k) const{ return k == numeric_limits<u64>::max() ? size() -
↳   less(x, k) : less(x, k + 1) - less(x, k); } // O(max_bit)
  int less(u64 x, u64 k) const{ // O(max_bit)
    x ^= tag;
    if(m < 63){ u64 high_x = x >> (m + 1), high_k = k >> (m + 1);
      if(high_x != high_k) return high_x < high_k ? size() : 0; }
    int cur = 0, ret = 0;
    for(int i = m;i >= 0;i--){ bool kb = (k >> i) & 1ull, xb = (x >> i) & 1ull;
      if(kb){ if(adj[cur][xb]) ret += pass[adj[cur][xb]];
        if(!adj[cur][xb ^ 1]) return ret; } else if(!adj[cur][xb]) return ret;
      cur = adj[cur][xb ^ kb]; }
```

XOR Trie / _xor_trie (계속)

```
    return ret; } };
```

07 Aho-Corasick / _ac

alphabet 필요. 기본 문자 집합은 소문자.

```
template <class Alphabet = alphabet::lowercase>
class _ac{ private: vector <array<int, Alphabet::size>> adj;
  vector<int> cnt, terminal_count, f, dep, dict, order; vector <bool> end;
  int new_node(){
    adj.push_back(array<int, Alphabet::size>{}); f.push_back(0); cnt.push_back(0);
↳     terminal_count.push_back(0);
    end.push_back(0); dict.push_back(0); dep.push_back(0); return (int)adj.size() - 1; }
public:
  class state{ private: friend class _ac;
    const _ac* owner; int idx;
    state(const _ac* owner, int idx) : owner(owner), idx(idx){}
  public: int id() const{ return idx; } // O(1)
    ll terminal_count() const{ return owner->terminal_count[idx]; } // O(1)
    ll match_count() const{ return owner->cnt[idx]; } // O(1)
    state go(int c) const{ return {owner, owner->adj[idx][c]}; } // O(1)
    int max_match_len() const{ // O(1)
      return owner->end[idx] ? owner->dep[idx] : (owner->dict[idx] ?
↳       owner->dep[owner->dict[idx]] : 0);
    }
    template<class F>
    void it_match(const F& f) const{ it_match(0x3f3f3f3f, f); } // O(matches)
    template<class F>
    void it_match(int k, const F& f) const{ // O(min(k, matches))
      if(k <= 0) return; int cnt = 0;
      if(owner->end[idx]){ f(state(owner, idx)); if(++cnt >= k) return; }
      for(int cur = owner->dict[idx];cur > 0 && cnt < k;cur = owner->dict[cur])
↳       f(state(owner, cur)), ++cnt;
    }
    template<class F>
    void it_match_lens(const F& f) const{ it_match_lens(0x3f3f3f3f, f); } // O(matches)
    template<class F>
    void it_match_lens(int k, const F& f) const{ // O(min(k, matches))
      it_match(k, [&](const state& matched){ f(owner->dep[matched.id()]); }); } };
  _ac(){ clear(); } // O(1)
  void clear(){ // O(1)
    order.clear(); adj.clear(); end.clear(); dep.clear();
    f.clear(); cnt.clear(); terminal_count.clear(); dict.clear(); new_node(); }
  state root() const{ return {this, 0}; } // O(1)
  const vector<int>& depth() const{ return dep; }
  const vector<bool>& terminal() const{ return end; }
  int size() const{ return (int)adj.size(); } // O(1)
  state insert(const string& s){ auto v = alphabet::tf<Alphabet>(s); return insert(v); }
↳   // O(|s|)
  state insert(span<const int> v){ // O(|v|)
    int cur = 0;
    for(auto &c : v){ int nxt = adj[cur][c];
      if(!nxt || dep[nxt] != dep[cur] + 1){ nxt = new_node(); adj[cur][c] = nxt;
        dep[nxt] = dep[cur] + 1; }
      cur = adj[cur][c]; }
    end[cur] = 1; terminal_count[cur]++; return {this, cur}; }
  void build(){ // O(max_node * Alphabet::size)
    cnt = terminal_count; fill(all(f), 0); fill(all(dict), 0);
```

Aho-Corasick / _ac (계속)

```
    for(int cur = 0; cur < (int)adj.size(); ++cur) for(int c = 0; c < Alphabet::size;
↳     ++c){
      int nxt = adj[cur][c]; if(nxt && dep[nxt] != dep[cur] + 1) adj[cur][c] = 0; }
    order.assign(1, 0);
    for(int c = 0;c < Alphabet::size;c++){ int nxt = adj[0][c]; if(!nxt) continue;
      f[nxt] = 0; dict[nxt] = 0; order.push_back(nxt); }
    for(int i = 1;i < (int)order.size();i++){ int cur = order[i];
      cnt[cur] += cnt[f[cur]]; dict[cur] = end[f[cur]] ? f[cur] : dict[f[cur]];
      for(int c = 0;c < Alphabet::size;c++){ int nxt = adj[cur][c];
        if(!nxt) adj[cur][c] = adj[f[cur]][c];
        else{ f[nxt] = adj[f[cur]][c]; order.push_back(nxt);
        } } }
  }
  vector<ll> count_each(const string& s) const{ auto v = alphabet::tf<Alphabet>(s);
↳   return count_each(v); } // O(|s| + max_node)
  vector<ll> count_each(span<const int> v) const{ // O(|v| + max_node)
    vector<ll> ret(size(), 0); it_state(v, [&](const state& st){ ret[st.id()]++; });
    for(int i = (int)order.size() - 1;i > 0;i--){
      int cur = order[i]; ret[f[cur]] += ret[cur]; }
    return ret; }
  ll count(const string& s) const{ auto v = alphabet::tf<Alphabet>(s); return count(v);
↳   } // O(|s|)
  ll count(span<const int> v) const{ // O(|v|)
    ll ret = 0; it_state(v, [&](const state& st){ ret += st.match_count(); });
    return ret; }
  template <class F>
  void it_state(const string& s, const F& f) const{ auto v = alphabet::tf<Alphabet>(s);
↳   it_state(v, f); } // O(|s|)
  template <class F>
  void it_state(span<const int> v, const F& f) const{ // O(|v|)
    auto cur = root();
    for(auto& c : v){ cur = cur.go(c); f(cur); }
  } };
```

07 KMP · kmp_policy / _kmp

```
template <class T>
class kmp_policy{ public: static void build(span<const T>){}
  static bool build_chk(span<const T> pat, int cur, int state){
    return pat[cur] == pat[state]; }
  static bool match_chk(span<const T> pat, span<const T> text, int idx, int state){
    return text[idx] == pat[state]; } };
template <class T = ll, class policy = kmp_policy<T>>
class _kmp{ // 0-based index
private: int m; vector <T> pat; vector <int> arr; policy cmp;
  void build(){ m = (int)pat.size(); cmp.build(span<const T>(pat));
    int nxt = 0; arr.assign(m, 0);
    for(int cur = 1;cur < m;cur++){
      while(nxt && !as_const(cmp).build_chk(span<const T>(pat), cur, nxt)) nxt = arr[nxt
↳       - 1];
      if(as_const(cmp).build_chk(span<const T>(pat), cur, nxt)) arr[cur] = ++nxt; } }
  static vector <T> tf(const string& s){ vector <T> ret; ret.reserve(s.size());
    for(unsigned char c : s) ret.push_back(T(c)); return ret; }
public: _kmp() : m(0){}
  _kmp(const string& s){ build(s); }
  _kmp(span<const T> v){ build(v); }
  void clear(){ pat.clear(); arr.clear(); m = 0; cmp.build(span<const T>(pat)); }
```

KMP · kmp_policy / _kmp (계속)

```
  void build(const string& s){ pat = tf(s); build(); }
  void build(span<const T> v){ pat.assign(all(v)); build(); }
  const vector<int>& fail() const{ return arr; } // O(1)
  template <class F>
  void it_match(const string& s, const F& f) const{ it_match(tf(s), f); } // O(|s| + m)
↳   checks
  template <class F>
  void it_match(span<const T> v, const F& f) const{ // O(|v| + m) checks
    int n = (int)v.size(); if(!m) return; int state = 0;
    for(int i = 0;i < n;i++){ state = go(state, v, i); if(state == m) f(i - m + 1);
    } }
  int go(int state, span<const T> v, int idx) const{ // O(m) checks, amortized O(1)
↳   checks
    if(!m) return 0; if(state == m) state = arr[m - 1];
    while(state && !cmp.match_chk(span<const T>(pat), v, idx, state)) state = arr[state
↳     - 1];
    if(cmp.match_chk(span<const T>(pat), v, idx, state)) state++; return state; }
  template <class F>
  void it_border(const F& f) const{ // O(m)
    if(!m) return; for(int x = arr[m - 1];x > 0;x = arr[x - 1]) f(x); }
};
```

07 Suffix Array · LCP / _sa

```
template <class T = ll>
class _sa{ private: vector <T> arr; vector <int> sa, pos, lcp, cnt, tmp_sa; int n, d;
  void build(){ if(!n) return; build_sa(); build_lcp(); }
  template <class F>
  void counting_sort(int sz, const F& f){ cnt.assign(sz, 0);
    for(int i = 0;i < n;i++) cnt[f(sa[i])]++;
    for(int i = 1;i < sz;i++) cnt[i] += cnt[i - 1];
    for(int i = n - 1;i >= 0;i--) tmp_sa[--cnt[f(sa[i])]] = sa[i]; sa.swap(tmp_sa); }
  void build_sa(){ vector <int> tmp(n); vector <pair<T, int>> num;
    for(int i = 0;i < n;i++) num.push_back({arr[i], i});
    sort(all(num)); pos[num[0].y] = 0; tmp_sa.resize(n); int r = 0;
    for(int i = 1; i < n; i++){ if(num[i].x != num[i - 1].x) r++; pos[num[i].y] = r;
    }
    for(d = 1; ;d <<= 1){ int sz = r + 1;
      counting_sort(sz + 1, [&](int i){ return i + d < n ? pos[i + d] + 1 : 0; });
      counting_sort(sz + 1, [&](int i){ return pos[i] + 1; });
      tmp[sa[0]] = 0; int nr = 0;
      for(int i = 1;i < n;i++){ int pre = sa[i - 1], cur = sa[i];
        int p1 = pos[pre], c1 = pos[cur]; int p2 = pre + d < n ? pos[pre + d] : -1;
        int c2 = cur + d < n ? pos[cur + d] : -1;
        if(p1 != c1 || p2 != c2) nr++; tmp[cur] = nr; }
      for(int i = 0;i < n;i++) pos[i] = tmp[i]; r = nr; if(r == n - 1) break; }
  }
  void build_lcp(){ int k = 0;
    for(int cur = 0;cur < n;cur++){ k = max(k - 1, 0); if(pos[cur] == n - 1) continue;
      int nxt = sa[pos[cur] + 1];
      while(cur + k < n && nxt + k < n && arr[cur + k] == arr[nxt + k]) k++;
      lcp[pos[cur]] = k; } }
public: _sa(){ clear(0); } // O(1)
  _sa(const string& s){ build(s); } // O(n log n)
  _sa(span<const T> v) { build(v); } // O(n log n)
  void build(const string& s){ clear(s.size()); for(unsigned char i : s)
↳   arr.push_back((T)i); build(); } // O(n log n)
```

Suffix Array · LCP / _sa (계속)

```
  void build(span<const T> v){ clear(v.size()); this->arr.assign(all(v)); build(); } //
↳   O(n log n)
  void clear(int n){ // O(n)
    this->n = n; d = 1; sa.assign(n, 0); arr.clear(); arr.reserve(n);
    iota(all(sa), 0); lcp.assign(n, 0); pos.assign(n, 0); }
  const vector<int>& get_sa() const{ return sa; } // O(1)
  const vector<int>& get_pos() const{ return pos; } // O(1)
  const vector<int>& get_lcp() const{ return lcp; } // O(1)
};
```

07 Manacher / _mnc

```
template <class T = ll>
class _mnc{ // 0-based index
private: vector <T> arr; vector <int> rad; int n = 0, m = 0; ll even = 0, odd = 0;
  void cal(){ // O(n)
    n = arr.size(); rad.assign(2 * n + 1, 0); even = odd = m = 0; int l = 0, r = -1;
    for(int i = 0;i < n;i++){ int k = 1;
      if(i <= r) k = min(rad[2 * (l + r - i) + 1] / 2 + 1, r - i + 1);
      while(i - k >= 0 && i + k < n && arr[i - k] == arr[i + k]) k++;
      rad[2 * i + 1] = 2 * k - 1; odd += k; m = max(m, 2 * k - 1);
      if(i + k - 1 > r){ l = i - k + 1; r = i + k - 1; } }
    l = 0; r = -1;
    for(int i = 0;i < n;i++){ int k = 0;
      if(i <= r) k = min(rad[2 * (l + r - i + 1)] / 2, r - i + 1);
      while(i - k - 1 >= 0 && i + k < n && arr[i - k - 1] == arr[i + k]) k++;
      rad[2 * i] = 2 * k; even += k; m = max(m, 2 * k);
      if(i + k - 1 > r){ l = i - k; r = i + k - 1; } } }
public: _mnc(){} // O(1)
  _mnc(const string& s){ build(s); } // O(n)
  _mnc(span<const T> v){ build(v); } // O(n)
  void build(const string& s){ // O(n)
    arr.clear(); arr.reserve(s.size()); for(unsigned char c : s) arr.push_back(c); cal();
  }
  void build(span<const T> v){ // O(n)
    arr.clear(); arr.reserve(v.size()); for(const auto& i : v) arr.push_back(i); cal();
  }
  const vector<int>& get_rad() const{ return rad; } // O(1)
  ll cnt() const{ return odd + even; } // O(1)
  ll cnt_odd() const{ return odd; } // O(1)
  ll cnt_even() const{ return even; } // O(1)
  int max_len() const{ return m; } // O(1)
  bool is_pal(int l, int r) const{ // O(1)
    int m = l + r + 1; return rad[m] >= r - l + 1; }
  pair<int, int> range_odd(int idx) const{ // O(1)
    int r = rad[2 * idx + 1] / 2; return { idx - r, idx + r }; }
  pair<int, int> range_even(int idx) const{ // O(1)
    int r = rad[2 * idx] / 2; return { idx - r, idx + r - 1 }; }
};
```

07 Z Algorithm / _z

```
template <class T = ll>
class _z { // 0-based index
private: vector<T> arr; vector<int> z; int n = 0;
  void build(){ n = (int)arr.size(); z.assign(n, 0); if(!n) return;
    z[0] = n; int l = 0, r = 0;
    for(int i = 1; i < n; i++){ if(i <= r) z[i] = min(r - i + 1, z[i - l]);
```

Z Algorithm / _z (계속)

```
      while(i + z[i] < n && arr[z[i]] == arr[i + z[i]]) z[i]++;
      if(i + z[i] - 1 > r){ l = i; r = i + z[i] - 1; } } }
  static vector<T> tf(const string& s){ vector<T> ret; ret.reserve(s.size());
    for(unsigned char c : s) ret.push_back((T)c); return ret; }
public: _z(){}
  _z(const string& s){ build(s); } // O(|s|)
  _z(span<const T> v){ build(v); } // O(|v|)
  void clear(){ arr.clear(); z.clear(); n = 0; } // O(1)
  void build(const string& s){ arr = tf(s); build(); } // O(|s|)
  void build(span<const T> v){ arr.assign(all(v)); build(); } // O(|v|)
  const vector<int>& result() const{ return z; } // O(1)
};
```

07 이중 Rolling Hash / _hash

_mint 필요 . 문자열 인덱스는 0-based.

```
template <int mod1 = 1000000007, int mod2 = 1000000009>
class _hash{ // 0-based index
private: using pi = pair<int, int>;
  template <int mod>
  class _core{ public: using mint = _mint<mod>;
    vector <int> pw, pre, ipw; mint key, inv; _core() = default;
    _core(int key) : key(key){ // O(n)
      assert(mod > 1); assert(this->key.v > 0); pre.clear(); pw.clear(); ipw.clear();
      pw.push_back(1); ipw.push_back(1); inv = this->key.inv(); }
    void ensure_pw(int k){ while((int)pw.size() <= k){
        pw.push_back((mint(pw.back()) * key).v);
        ipw.push_back((mint(ipw.back()) * inv).v); } }
    int size() const{ return (int)pre.size(); }
    void push_back(int v){ // O(1)
      mint now = v; ensure_pw(size()); if(pre.empty()) pre.push_back(now.v);
      else pre.push_back((mint(pre.back()) + now * pw[size()]).v); }
    void pop_back(){ // O(1)
      assert(!pre.empty()); pre.pop_back(); }
    ll ret(int l, int r) const{ // O(1)
      mint ret = pre[r]; if(l > 0) ret -= pre[l - 1]; ret *= ipw[l];
      return ret.v; } }; pair <_core<mod1>, _core<mod2>> arr;
public: _hash(pi key = {911382323, 972663749}){ clear(key); }
  void clear(pi key = {911382323, 972663749}){ // O(1)
    arr.x = {key.x}; arr.y = {key.y}; }
  void reset(){ arr.x.pre.clear(); arr.y.pre.clear(); }
  void push_back(int v){ arr.x.push_back(v); arr.y.push_back(v); } // O(1)
  void pop_back(){ arr.x.pop_back(); arr.y.pop_back(); } // O(1)
  int size() const{ return arr.x.size(); } // O(1)
  ll ret(int l, int r) const{ // O(1)
    if(l > r) return 0; return ((ll)arr.x.ret(l, r) << 32) + arr.y.ret(l, r); }
  bool same(int l1, int r1, int l2, int r2) const{ // O(1)
    if(r1 - l1 != r2 - l2) return false; if(l1 > r1) return true;
    return arr.x.ret(l1, r1) == arr.x.ret(l2, r2) && arr.y.ret(l1, r1) == arr.y.ret(l2,
↳     r2);
  } };
```

08 CCW / _ccw

```
class _ccw{ public: _ccw() {}
  ll ret(ll x1, ll y1, ll x2, ll y2, ll x3, ll y3){
    return ret({x1, y1}, {x2, y2}, {x3, y3}); }
  ll ret(pll v1, pll v2, pll v3){ i128 num = ((i128)v2.x - v1.x) * ((i128)v3.y - v1.y);
```

CCW / _ccw (계속)

```
    num -= ((i128)v3.x - v1.x) * ((i128)v2.y - v1.y); if(num > 0) return 1;
    else if(num < 0) return -1; return 0; }
};
```

08 편각 정렬 / _angle_sort

```
template <typename T = pll>
class _angle_sort{ private: using i128 = __int128_t;
  static i128 dx(const T& p, ll cx){ return (i128)p.x - cx; }
  static i128 dy(const T& p, ll cy){ return (i128)p.y - cy; }
  static int half_ccw(i128 x, i128 y){ return (y > 0 || (y == 0 && x >= 0)) ? 0 : 1; }
  static int half_cw(i128 x, i128 y){ return (y < 0 || (y == 0 && x >= 0)) ? 0 : 1; }
public: static void sort(span<T> arr, ll cx = 0, ll cy = 0, bool is_clock = false){
    std::sort(arr.begin(), arr.end(), [&](const T& a, const T& b){
      i128 ax = dx(a, cx), ay = dy(a, cy); i128 bx = dx(b, cx), by = dy(b, cy);
      int ha = is_clock ? half_cw(ax, ay) : half_ccw(ax, ay);
      int hb = is_clock ? half_cw(bx, by) : half_ccw(bx, by);
      if(ha != hb) return ha < hb; i128 cross = ax * by - ay * bx;
      if(cross) return is_clock ? cross < 0 : cross > 0; i128 da = ax * ax + ay * ay;
      i128 db = bx * bx + by * by; return da < db; });
  }
  static bool is_same(const T& a, const T& b, ll cx = 0, ll cy = 0){
    i128 ax = dx(a, cx), ay = dy(a, cy); i128 bx = dx(b, cx), by = dy(b, cy);
    return ax * by - ay * bx == 0 && ax * bx + ay * by > 0; } };
```

08 선분 교차 / _li

```
class _li{ public: static int ccw(const pll& v1, const pll& v2, const pll& v3){ // O(1)
    i128 x1 = (i128)v2.x - v1.x, y1 = (i128)v2.y - v1.y;
    i128 x2 = (i128)v3.x - v1.x, y2 = (i128)v3.y - v1.y; i128 cross = x1 * y2 - x2 * y1;
    if(cross > 0) return 1; else if(cross < 0) return -1; return 0;
  }
  static bool intersect(pair<pll, pll> a, pair<pll, pll> b){ // O(1)
    return intersect(a.x, a.y, b.x, b.y); }
  static bool intersect(pll av1, pll av2, pll bv1, pll bv2){ // O(1)
    int ac = ccw(av1, av2, bv1) * ccw(av1, av2, bv2);
    int bc = ccw(bv1, bv2, av1) * ccw(bv1, bv2, av2);
    if(!ac && !bc){ if(av1 > av2) swap(av1, av2); if(bv1 > bv2) swap(bv1, bv2);
      return bv1 <= av2 && av1 <= bv2; }
    return ac <= 0 && bc <= 0; } };
```

08 유리수 / _frac

```
class _frac{ public: i128 p, q;
  _frac(i128 p = 0, i128 q = 1){ set(p, q); }
  void set(i128 p, i128 q){ assert(q);
    if(q < 0){ p = -p; q = -q; }
    if(!p){ this->p = 0; this->q = 1; return; }
    i128 g = gcd(p < 0 ? -p : p, q); this->p = p / g; this->q = q / g; }
  ld to_ld() const{ return (ld)p / (ld)q; }
  i128 floor() const{ if(p >= 0) return p / q; return -((-p + q - 1) / q);
  }
  i128 ceil() const{ if(p >= 0) return (p + q - 1) / q; return -((-p) / q);
  }
  _frac operator-() const{ return _frac(-p, q); }
  _frac inv() const{ assert(p != 0); return _frac(q, p); }
  friend bool operator==(const _frac& a, i128 b){ assert(a.q); i128 p = a.p, q = a.q;
    if(q < 0){ p = -p; q = -q; }
```

유리수 / _frac (계속)

```
    if(p % q) return false; return (p / q) == b; }
  friend bool operator==(i128 a, const _frac& b){ return b == a; }
  friend bool operator!=(const _frac& a, i128 b){ return !(a == b); }
  friend bool operator!=(i128 a, const _frac& b){ return !(a == b); }
  friend bool operator==(const _frac& a, const _frac& b){ return a.p == b.p && a.q ==
↳   b.q; }
  friend bool operator!=(const _frac& a, const _frac& b){ return !(a == b); }
  friend bool operator<(const _frac& a, const _frac& b){ return a.p * b.q < b.p * a.q; }
  friend bool operator>(const _frac& a, const _frac& b){ return b < a; }
  friend bool operator<=(const _frac& a, const _frac& b){ return !(b < a); }
  friend bool operator>=(const _frac& a, const _frac& b){ return !(a < b); }
  friend _frac operator+(const _frac& a, const _frac& b){ return _frac(a.p * b.q + b.p *
↳   a.q, a.q * b.q); }
  friend _frac operator-(const _frac& a, const _frac& b){ return _frac(a.p * b.q - b.p *
↳   a.q, a.q * b.q); }
  friend _frac operator*(const _frac& a, const _frac& b){ return _frac(a.p * b.p, a.q *
↳   b.q); }
  friend _frac operator/(const _frac& a, const _frac& b){ assert(b.p != 0); return
↳   _frac(a.p * b.q, a.q * b.p); }
};
```

08 Convex Hull / _ch

```
class _ch{ public: _ch() {}
  vector <ll> num; vector <pll> p; ll cnt = 0;
  class node{ public: ll x, y, idx;
    node(pll a, ll idx) : node(a.x, a.y, idx) {};
    node(ll x, ll y, ll idx){ this->x = x; this->y = y; this->idx = idx;
    }
    operator pll() const{ return { x, y }; } }; vector <node> arr, st;
  ll dist(pll v1, pll v2){ ll dx = v1.x - v2.x, dy = v1.y - v2.y;
    return dx * dx + dy * dy; }
  ll ccw(pll v1, pll v2, pll v3){ i128 ret = ((i128)v2.x - v1.x) * ((i128)v3.y - v1.y);
    ret -= ((i128)v3.x - v1.x) * ((i128)v2.y - v1.y); return (ret > 0) - (ret < 0); }
  void add(ll a, ll b){ add({a, b}); }
  void add(pll a){ arr.push_back({a, cnt++}); }
  void build(){ // O(n log n)
    st.clear(); p.clear(); auto v = arr;
    sort(all(v), [&](const node& a, const node& b){
      return tie(a.x, a.y, a.idx) < tie(b.x, b.y, b.idx); });
    v.erase(unique(all(v), [&](const node& a, const node& b){
      return a.x == b.x && a.y == b.y; }), v.end()); if(v.empty()) return;
    for(const auto& i : v){
      while(st.size() >= 2 && ccw(st[st.size() - 2], st.back(), i) <= 0) st.pop_back();
      st.push_back(i); }
    ll sz = st.size();
    for(ll i = (ll)v.size() - 2;i >= 0;i--){
      while(st.size() > sz && ccw(st[st.size() - 2], st.back(), v[i]) <= 0)
↳       st.pop_back();
      st.push_back(v[i]); }
    if(st.size() > 1) st.pop_back(); for(const auto& i : st) p.push_back(i); }
  const vector <pll>& ret() const{ return p; } // O(1)
  const vector <node>& idx() const{ return st; } // O(1)
};
```

08 모듈러 행렬 / _mod_matrix

_mint 필요. 행렬 인덱스는 0-based.

```
template <u64 mod = 998244353>
class _mod_matrix{ // 0-based index, Require _mint
public: using mint = _mint<mod>; int n, m; vector <mint> arr;
  _mod_matrix() : n(0), m(0){}
  _mod_matrix(int n, int m) : n(n), m(m), arr(1LL * n * m){}
  mint& at(int i, int j){ return arr[1LL * i * m + j]; }
  const mint& at(int i, int j) const{ return arr[1LL * i * m + j]; }
  void add(int i, int j, ll v){ at(i, j) += v; }
  void set(int i, int j, ll v){ at(i, j) = v; }
  static _mod_matrix id(int n){ _mod_matrix ret(n, n);
    for(int i = 0; i < n; i++) ret.at(i, i) = 1; return ret; }
  _mod_matrix operator*(const _mod_matrix& ot) const{ assert(m == ot.n);
    _mod_matrix ret(n, ot.m);
    for(int i = 0; i < n; i++){ for(int k = 0; k < m; k++){ const mint v = at(i, k);
        if(!v.v) continue; for(int j = 0; j < ot.m; j++) ret.at(i, j) += v * ot.at(k, j);
      } }
    return ret; }
  _mod_matrix operator+(const _mod_matrix& ot) const{ assert(n == ot.n && m == ot.m);
    _mod_matrix ret(n, m);
    for(int i = 0; i < n * m; i++) ret.arr[i] = arr[i] + ot.arr[i]; return ret; }
  _mod_matrix pow(ll k) const{ // n*n matrix
    assert(n == m); assert(k >= 0); _mod_matrix base = *this, ret = id(n);
    while(k > 0){ if(k & 1) ret = ret * base; base = base * base;
      k >>= 1LL; }
    return ret; }
  _mod_matrix transpose() const{ _mod_matrix ret(m, n);
    for(int i = 0;i < n;i++) for(int j = 0;j < m;j++) ret.at(j, i) = at(i, j);
    return ret; }
  friend ostream& operator<<(ostream& out, const _mod_matrix& cur){
    for(int i = 0; i < cur.n; i++){
      for(int j = 0; j < cur.m; j++) out << cur.at(i, j) << " "; out << "\n"; }
    return out; } };
```

08 다각형 넓이 / _area

```
class _area{ public: _area() {}
  ll ret(span<const pll> v){ // div by 2
    i128 num = 0;
    for(int i = 0;i < v.size();i++){ auto [x1, y1] = v[i];
      auto [x2, y2] = v[(i + 1) % v.size()]; num += (i128)x1 * y2 - (i128)x2 * y1; }
    return (ll)abs(num); } };
```

08 직선 방정식 · 교점 / _line

_frac 필요.

```
class _line{ public: _frac a, b; i128 x = 0; bool vert = false;
  _line() = default;
  _line(const pll& p1, const pll& p2){ if(p1.x == p2.x){ vert = true; x = p1.x;
      a = _frac(), b = _frac(); } else{
      vert = false; ll dx = p2.x - p1.x, dy = p2.y - p1.y;
      a = _frac(dy, dx), b = _frac(p1.y) - a * _frac(p1.x); } }
  _line(const _frac& slope, const pll& p){ vert = false; x = 0;
    a = slope; b = _frac(p.y) - a * _frac(p.x); }
  _frac f(const _frac& x) const{ assert(!vert); return a * x + b;
  }
  bool intersect(const _line& ot, pair<_frac, _frac>& ret) const{
    if(vert && ot.vert) return false;
```

직선 방정식 · 교점 / _line (계속)

```
    if(vert){ _frac p1 = _frac(x), p2 = ot.f(p1); ret = {p1, p2}; return true;
    }
    if(ot.vert){ _frac p1 = _frac(ot.x), p2 = f(p1); ret = {p1, p2}; return true;
    }
    if(a == ot.a) return false;
    _frac p1 = (ot.b - b) / (a - ot.a), p2 = f(p1); ret = {p1, p2}; return true; }
  bool same(const _line& ot) const{ if(vert != ot.vert) return false;
    if(vert) return x == ot.x; return a == ot.a && b == ot.b; }
};
```

08 점의 다각형 포함 여부 / _inter

_li 필요. 경계 포함, 원본 구현의 ray 끝점은 x=1e12.

```
class _inter{ // Require line_intersection
public: static bool ret(span<const pll> p, pll now){ ll cnt = 0, flag = 0, sz = p.size();
    if(sz < 3) return 0; pll nxt = {1e12, now.y + 1};
    for(int i = 0;i < sz;i++){
      cnt += _li::intersect({p[i], p[(i + 1) % sz]}, {now, nxt}); }
    for(int i = 0;i < sz;i++){
      flag += _li::intersect({p[i], p[(i + 1) % sz]}, {now, now}); }
    return (cnt % 2 | flag); } };
```

08 Rotating Calipers / _rc

_ch 필요. 볼록 껍질 순서의 정점을 입력한다 .

```
class _rc{ public: using node = _ch::node;
private: vector <node> p; pair <pll, pll> result{}; pll ids = {-1, -1};
  ll mx = 0; ld len = 0;
  i128 area(ll a, ll b, ll c) const{
    i128 ret = ((i128)p[b].x - p[a].x) * ((i128)p[c].y - p[a].y);
    ret -= ((i128)p[c].x - p[a].x) * ((i128)p[b].y - p[a].y);
    return ret < 0 ? -ret : ret; }
  void update(ll l, ll r){ ll dx = p[l].x - p[r].x, dy = p[l].y - p[r].y;
    ll now = dx * dx + dy * dy; if(now <= mx) return;
    mx = now; result = {pll(p[l]), pll(p[r])}; ids = {p[l].idx, p[r].idx}; }
public: void build(span<const node> arr){ p.assign(all(arr)); build(); } // O(n)
  void build(span<const pll> arr){ // O(n)
    p.clear(); p.reserve(arr.size());
    for(ll i = 0;i < arr.size();i++) p.push_back({arr[i], i}); build(); }
  void build(){ // O(n)
    vector <node> v;
    for(const auto& i : p){
      if(v.empty() || v.back().x != i.x || v.back().y != i.y) v.push_back(i); }
    if(v.size() > 1 && v.front().x == v.back().x && v.front().y == v.back().y)
↳     v.pop_back();
    p.swap(v); mx = 0; len = 0; result = {}; ids = {-1, -1};
    ll sz = p.size(); if(!sz) return;
    result = {pll(p[0]), pll(p[0])}; ids = {p[0].idx, p[0].idx}; if(sz == 1) return;
    bool line = 1;
    for(ll i = 2;i < sz;i++) if(area(0, 1, i)){ line = 0; break; }
    if(line){ ll l = 0, r = 0;
      for(ll i = 1;i < sz;i++){ if(tie(p[i].x, p[i].y) < tie(p[l].x, p[l].y)) l = i;
        if(tie(p[i].x, p[i].y) > tie(p[r].x, p[r].y)) r = i; }
      update(l, r); } else{
      ll r = 1;
      for(ll l = 0;l < sz;l++){ ll nxt = (l + 1) % sz; r = max(r, l + 1);
        while(r + 1 < l + sz && area(l, nxt, (r + 1) % sz) >= area(l, nxt, r % sz)){
          update(l, r % sz); update(nxt, r % sz); r++; }
```

Rotating Calipers / _rc (계속)

```
        update(l, r % sz); update(nxt, r % sz); } }
    len = sqrtl(mx); }
  const pair <pll, pll>& ret() const{ return result; } // O(1)
  const pll& idx() const{ return ids; } // O(1)
  ll dist2() const{ return mx; } // O(1)
  ld dist() const{ return len; } // O(1)
};
```

08 벡터 연산 / _dot

```
class _dot{ public: _dot() {}
  // ret > 0 -> angle < 90, ret < 0 -> angle > 90, ret == 0 -> angle == 90
  ll ret(ll x1, ll y1, ll x2, ll y2, ll x3, ll y3){
    return ret({x1, y1}, {x2, y2}, {x3, y3}); }
  ll ret(pll l, pll m, pll r){ pll a = {m.x - l.x, m.y - l.y};
    pll b = {m.x - r.x, m.y - r.y}; return a.x * b.x + a.y * b.y; }
};
```

08 GF(2) 선형대수 / _gf2

```
template <size_t m>
class _gf2{ // 0-based index
public: enum class state{ inconsistent, unique, multiple };
  class result{ public: state status;
    bitset <m> value; ll rk, fr;
    bool inconsistent() const{ return status == state::inconsistent; } // O(1)
    bool unique() const{ return status == state::unique; } // O(1)
    bool multiple() const{ return status == state::multiple; } // O(1)
    const bitset <m>& get_value() const{ return value; } // O(1)
    ll rank() const{ return rk; } // O(1)
    ll free_count() const{ return fr; } // O(1)
  }; vector <bitset <m + 1>> bs;
private: result ans{};
public: _gf2(ll n = 0) : bs(n){}
  void set_a(ll cy, ll cx, bool v){ bs[cy][cx] = v; }
  void set_b(ll cy, bool v){ bs[cy][m] = v; }
  void clear(){ for(auto& i : bs) i.reset(); }
  void build(){ ll n = bs.size(); auto arr = bs; vector <ll> pos(m, -1);
    ll cy = 0;
    for(ll cx = 0;cx < m && cy < n;cx++){ ll now = -1;
      for(ll i = cy;i < n;i++){ if(!arr[i][cx]) continue; now = i; break;
      }
      if(now == -1) continue; swap(arr[now], arr[cy]); pos[cx] = cy;
      for(ll i = 0;i < n;i++){ if(i == cy || !arr[i][cx]) continue; arr[i] ^= arr[cy];
      }
      cy++; }
    ans = {cy == m ? state::unique : state::multiple, {}, cy, (ll)m - cy};
    for(ll i = cy;i < n;i++){ if(!arr[i][m]) continue;
      ans.status = state::inconsistent; return; }
    for(ll i = 0;i < m;i++) if(pos[i] != -1) ans.value[i] = arr[pos[i]][m]; }
  const result& ret() const{ return ans; } // O(1)
};
```

08 XOR Basis / _xor_basis

```
template <size_t N = 63>
class _xor_basis{ public: array<u64, N> basis{}; int rk = 0;
  _xor_basis(){ clear(); } // O(max_bit)
  void clear(){ basis.fill(0); rk = 0; } // O(max_bit)
  bool insert(u64 x){ // O(max_bit)
    for(int i = (int)N - 1; i >= 0; i--){ if(((x >> i) & 1ull) == 0) continue;
      if(!basis[i]){ basis[i] = x; rk++; return 1;
      }
      x ^= basis[i]; }
    return 0; }
  bool can_make(u64 x) const{ // O(max_bit)
    for(int i = (int)N - 1; i >= 0; i--){ if(((x >> i) & 1ull) == 0) continue;
      if(!basis[i]) return 0; x ^= basis[i]; }
    return 1; }
  u64 max_xor(u64 x = 0) const{ // O(max_bit)
    u64 ret = x; for(int i = (int)N - 1; i >= 0; i--) ret = max(ret, ret ^ basis[i]);
    return ret; }
  u64 min_xor(u64 x = 0) const{ // O(max_bit)
    u64 ret = x; for(int i = (int)N - 1; i >= 0; i--) ret = min(ret, ret ^ basis[i]);
    return ret; }
  int rank() const{ return rk; } // O(1)
};
```

09 FFT / _fft

sz는 결과 계수 개수 이상. double 기반이므로 정밀도 확인.

```
template <size_t sz>
class _fft{ private: using cpd = complex <double>;
  static constexpr size_t n = sz ? bit_ceil(sz) : 0;
  static void transform(vector <cpd>& v, bool inv){ // O(n log n)
    if(n < 2) return;
    for(ll i = 1, j = 0;i < n;i++){ ll bit = n >> 1; for(;j & bit;bit >>= 1) j ^= bit;
      j ^= bit; if(i < j) swap(v[i], v[j]); }
    const double PI = acos(-1.0);
    for(ll k = 1;k < n;k <<= 1){ double a = PI / k * (inv ? 1 : -1);
      cpd w(cos(a), sin(a));
      for(ll i = 0;i < n;i += 2 * k){ cpd wp(1, 0);
        for(ll j = 0;j < k;j++){ cpd l = v[i + j], r = v[i + j + k] * wp;
          v[i + j] = l + r; v[i + j + k] = l - r; wp *= w;
        } } }
    if(inv) for(auto& i : v) i /= (double)n; }
public:
  class node{ friend class _fft;
  private: vector <cpd> v; ll len;
    node(ll len) : v(n), len(len){}
  public: node& operator+=(const node& a){ // O(n)
      for(ll i = 0;i < n;i++) v[i] += a.v[i]; len = max(len, a.len); return *this;
    }
    node& operator*=(const node& a){ // O(n)
      for(ll i = 0;i < n;i++) v[i] *= a.v[i]; len = len && a.len ? len + a.len - 1 : 0;
      return *this; }
    friend node operator+(node a, const node& b){ a += b; return a; } // O(n)
    friend node operator*(node a, const node& b){ a *= b; return a; } // O(n)
    friend ostream& operator<<(ostream& out, const node& a){ // O(n log n)
      auto ret = _fft{}.inverse(a); if(ret.empty()) return out << 0;
      for(ll i = (ll)ret.size() - 1;i >= 0;i--) out << ret[i]; return out; }
  };
```

FFT / _fft (계속)

```
  template <integral T>
  node make(const vector <T>& arr) const{ // O(n log n)
    node ret(arr.size()); for(ll i = 0;i < (ll)arr.size();i++) ret.v[i] = (double)arr[i];
    transform(ret.v, 0); return ret; }
  template <integral T = ll>
  vector <T> inverse(const node& a) const{ // O(n log n)
    if(!a.len) return {}; auto v = a.v; transform(v, 1); vector <T> ret(a.len);
    for(ll i = 0;i < a.len;i++) ret[i] = (T)round(v[i].real()); return ret; }
  node carry(const node& a, ll radix = 10) const{ // O(n log n)
    auto ret = inverse(a); if(ret.empty()) return a; i128 c = 0;
    for(auto& i : ret){ c += i; i = (ll)(c % radix);
      c /= radix; }
    while(c){ ret.push_back((ll)(c % radix)); c /= radix;
    }
    while(ret.size() > 1 && !ret.back()) ret.pop_back(); return make(ret); }
};
```

10 정수·실수 이분 탐색 / _bs, _fbs

```
class _bs { // integer
public:
  template <class F>
  static ll max_right(ll l, ll r, const F& f) { ll lo = l, hi = r, mid = -1;
    while (hi > lo) { mid = midpoint(hi, lo); if (f(mid)) lo = mid;
      else hi = mid - 1; }
    return lo; }
  template <class F>
  static ll min_left(ll l, ll r, const F& f) { ll lo = l, hi = r, mid = -1;
    while (hi > lo) { mid = midpoint(lo, hi); if (f(mid)) hi = mid;
      else lo = mid + 1; }
    return lo; } };
class _fbs { // real
public:
  template <class F>
  static ld max_right(ld l, ld r, const F& f, int iter = 80) { ld lo = l, hi = r;
    for(int i = 0; i < iter; i++){ ld mid = (lo + hi) / 2; if (f(mid)) lo = mid;
      else hi = mid; }
    return lo; }
  template <class F>
  static ld min_left(ld l, ld r, const F& f, int iter = 80) { ld lo = l, hi = r;
    for(int i = 0; i < iter; i++){ ld mid = (lo + hi) / 2; if (f(mid)) hi = mid;
      else lo = mid; }
    return lo; } };
```

10 정수·실수 삼분 탐색 / _ts, _fts

```
class _ts { // integer
public:
  template <class F>
  static ll max(ll l, ll r, const F& f) { while ((i128)r - l > 3) {
      ll st = (2 * (i128)l + r) / 3, en = ((i128)l + 2 * (i128)r) / 3;
      if(f(st) < f(en)) l = st; else r = en; }
    ll result = MINF;
    for(i128 cur = l; cur <= r;cur++) result = std::max(result, f((ll)cur));
    return result; }
  template <class F>
  static ll min(ll l, ll r, const F& f) { while ((i128)r - l > 3) {
      ll st = (2 * (i128)l + r) / 3, en = ((i128)l + 2 * (i128)r) / 3;
```

정수·실수 삼분 탐색 / _ts, _fts (계속)

```
      if(f(st) > f(en)) l = st; else r = en; }
    ll result = INF;
    for(i128 cur = l; cur <= r;cur++) result = std::min(result, f((ll)cur));
    return result; } };
class _fts { // real
public:
  template <class F>
  static ld max(ld l, ld r, const F& f, int iter = 120) { for(int i = 0; i < iter; i++){
      ld st = (2 * l + r) / 3, en = (l + 2 * r) / 3; if(f(st) < f(en)) l = st;
      else r = en; }
    return std::max(f(l), f(r)); }
  template <class F>
  static ld min(ld l, ld r, const F& f, int iter = 120) { for(int i = 0; i < iter; i++){
      ld st = (2 * l + r) / 3, en = (l + 2 * r) / 3; if(f(st) > f(en)) l = st;
      else r = en; }
    return std::min(f(l), f(r)); } };
```

10 LCS / _lcs

```
class _lcs{ private: string a, b;
  vector <vector<ll>> dp; ll n, m;
public: _lcs() : _lcs("", ""){}
  _lcs(const string& a, const string& b){ build(a, b); }
  void build(const string& s, const string& t){ a = s; b = t;
    n = a.size(); m = b.size(); dp.assign(n + 1, vector<ll>(m + 1, 0));
    for(int i = 0;i < n;i++){ for(int j = 0;j < m;j++){ if(a[i] == b[j]){
          dp[i + 1][j + 1] = dp[i][j] + 1; } else{
          dp[i + 1][j + 1] = max(dp[i][j + 1], dp[i + 1][j]); } }
    } }
  ll ret() const{ return dp[n][m]; } // lcs 길이 반환
  string track() const{ // lcs string 반환
    string ret; ll cy = n, cx = m;
    while(cy && cx){ if(a[cy - 1] == b[cx - 1]){ ret.push_back(a[cy - 1]); cy--; cx--;
      } else if(dp[cy - 1][cx] > dp[cy][cx - 1]) cy--; else cx--;
    }
    reverse(ret.begin(), ret.end()); return ret; }
};
```

10 Mo's Algorithm / _mos

```
class mos_policy{ public:
  class node{ public: ll v;
    node(ll v) : v(v){} };
  class ret{ public: ll v;
    ret(ll v) : v(v){} };
  static node leaf(){ return {0}; }
  static ret id(){ return {0}; }
  static void reset(){}
  static void push_front(int idx, const node& v, ret& cur){}
  static void push_back(int idx, const node& v, ret& cur){ push_front(idx, v, cur); }
  static void pop_front(int idx, const node& v, ret& cur){}
  static void pop_back(int idx, const node& v, ret& cur){ pop_front(idx, v, cur); }
};
template <class policy = mos_policy>
class _mos{ bool base;
public: using node = typename policy::node; using ret  = typename policy::ret;
private: int n, b, cl = 0, cr = -1; ret cur = id();
  static void push_front(int idx, const node& v, ret& cur){ policy::push_front(idx, v,
```

Mo’s Algorithm / _mos (계속)

```
↳   cur); }
  static void push_back(int idx, const node& v, ret& cur){ policy::push_back(idx, v,
↳   cur); }
  static void pop_front(int idx, const node& v, ret& cur){ policy::pop_front(idx, v,
↳   cur); }
  static void pop_back(int idx, const node& v, ret& cur){ policy::pop_back(idx, v, cur);
↳   }
  static node leaf(){ return policy::leaf(); }
  static ret id(){ return policy::id(); }
  class query{ public: int l, r, idx;
  };
  class cmp{ public: int b;
    bool operator()(const query& a, const query& c) const {
      int ab = a.l / b, cb = c.l / b; if(ab != cb) return ab < cb;
      if(ab & 1) return a.r > c.r; return a.r < c.r; }
  };
  void apply(int nl, int nr){ while(cl > nl){ --cl; push_front(cl, arr[cl], cur); }
    while(cr < nr){ ++cr; push_back(cr, arr[cr], cur); }
    while(cl < nl){ pop_front(cl, arr[cl], cur); ++cl; }
    while(cr > nr){ pop_back(cr, arr[cr], cur); --cr; } }
public: vector <ret> result; vector <node> arr; vector <query> vq;
  _mos(int n = 0, int b = 0, bool base = 1){ clear(n, b, base); } // O(n)
  void clear(int n, int b = 0, bool base = 1){ // O(n)
    this->n = n; this->base = base; this->b = b;
    if(!b) this->b = sqrt(n + 1); vq.clear(); arr.assign(n + 1, leaf()); result.clear();
    cl = base; cr = base - 1; cur = id(); }
  const vector<ret>& get_result() const{ return result; } // O(1)
  void add(int l, int r){ vq.push_back({l, r, (int)vq.size()}); } // O(1)
  void set(int idx, const node& v){ if(idx >= base && idx <= n) arr[idx] = v; } // O(1)
  void build(){ // O(q log q + (n^2 / b + qb) * update)
    policy::reset(); cl = base; cr = base - 1; cur = id();
    if (vq.empty()) { result.clear(); return; }
    sort(all(vq), cmp{b}); result.assign(vq.size(), id());
    for(const auto& cq : vq){ apply(cq.l, cq.r); result[cq.idx] = cur;
    } } };
```

10 Convex Hull Trick / _cht

기울기 : min 비증가 / max 비감소 . query_monotone 의 x는 비감소 .

```
class _cht{ public:
  class line{ public: ll a, b; line(){}
    line(ll a, ll b) : a(a), b(b){}
    ll f(ll x) const{ return (ll)((i128)a * x + b); } };
  line id_line() const{ return is_min ? line(0, INF) : line(0, MINF); }
  vector<line> v; bool is_min = 1; ll idx = 0;
  _cht(bool is_min = 1) : is_min(is_min){}
  void clear(bool is_min = 1){ this->is_min = is_min; v.clear(); idx = 0; }
  bool chk(const line& a, const line& b, const line& c) const{
    i128 x1 = ((i128)a.b - b.b) * ((i128)c.a - b.a);
    i128 x2 = ((i128)b.b - c.b) * ((i128)b.a - a.a); return x1 >= x2; }
  bool better(const line& a, const line& b, ll x) const{
    i128 y1 = (i128)a.a * x + a.b, y2 = (i128)b.a * x + b.b;
    return is_min ? y1 <= y2 : y1 >= y2; }
  bool better_same(const line& a, const line& b) const{
    return is_min ? a.b <= b.b : a.b >= b.b; }
  // 기울기 a: 최소 모드에서는 비증가, 최대 모드에서는 비감소 순서
  void add(line cur){ // O(1) amortized
```

Convex Hull Trick / _cht (계속)

```
    if(!v.empty() && v.back().a == cur.a){ if(!better_same(cur, v.back())) return;
      v.pop_back(); }
    while(v.size() >= 2 && chk(v[v.size() - 2], v.back(), cur)) v.pop_back();
    v.push_back(cur); if(idx >= (ll)v.size()) idx = (ll)v.size() - 1; }
  // x는 비감소 순서로 전달하며, clear 후에는 순서를 새로 시작
  line query_monotone(ll x){ // O(1) amortized
    if(v.empty()) return id_line();
    while(idx + 1 < (ll)v.size() && better(v[idx + 1], v[idx], x)) idx++; return v[idx];
  }
  line query(ll x) const{ // O(log n)
    if(v.empty()) return id_line(); ll lo = 0, hi = (ll)v.size() - 1;
    while(lo < hi){ ll mid = (lo + hi) >> 1ll;
      if(better(v[mid + 1], v[mid], x)) lo = mid + 1; else hi = mid; }
    return v[lo]; } };
```

10 Parallel Binary Search / _pbs

```
class pbs_policy{ public:
  class query{ public: ll v;
    query(ll v) : v(v){} };
  static void reset(){}
  static void apply(ll t){}
  static bool check(const query& q){ return 0; } };
template<class policy = pbs_policy>
class _pbs{ public: using query = typename policy::query;
  ll lo, hi; vector<query> q;
private: vector<ll> result;
public: _pbs(ll lo = 0, ll hi = -1){ clear(lo, hi); } // O(1)
  void clear(ll lo = 0, ll hi = -1){ // O(q)
    this->lo = lo; this->hi = hi; q.clear(); result.clear(); }
  int add(query a){ // O(1) amortized
    int id = q.size(); q.push_back(move(a)); return id; }
  const vector<ll>& get_result() const{ return result; } // O(1)
  void build(bool first = 1){ // O(log(w + 1) * (reset + w * apply + q * check + w + q))
    result.assign(q.size(), first ? hi + 1 : lo - 1); if(q.empty() || lo > hi) return;
    ll w = hi - lo + 1; vector<ll> l(q.size(), 0), r(q.size(), w - 1);
    vector<vector<int>> bucket(w);
    while(1){ for(auto& cur : bucket) cur.clear(); bool pending = 0;
      for(int i = 0; i < (int)q.size(); i++){ if(l[i] > r[i]) continue;
        ll mid = midpoint(l[i], r[i]); bucket[mid].push_back(i); pending = 1; }
      if(!pending) break; policy::reset();
      for(ll mid = 0; mid < w; mid++){ ll t = lo + mid; policy::apply(t);
        for(int id : bucket[mid]){ bool ok = policy::check(q[id]); if(ok) result[id] = t;
          if(ok == first) r[id] = mid - 1; else l[id] = mid + 1; }
      } } }
};
```